

# Lecture 14

## Generative Models: VAE and GAN

Speaker: Kaiming He

# Overview

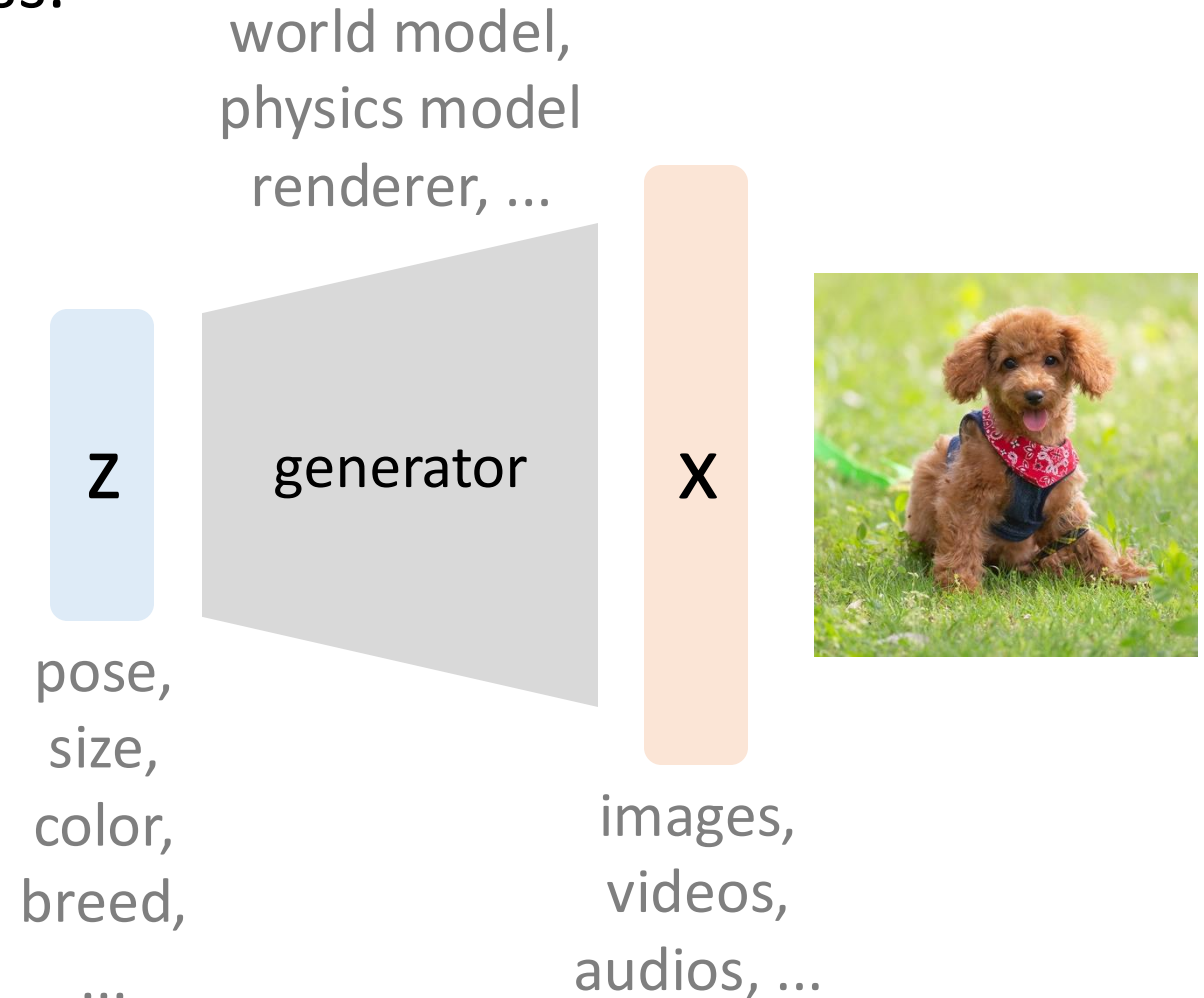
- Latent Variable Models
- Variational Autoencoder (VAE)
- Generative Adversarial Network (GAN)

# **Latent Variable Models**

# Latent Variable Models

Assuming a data generation process:

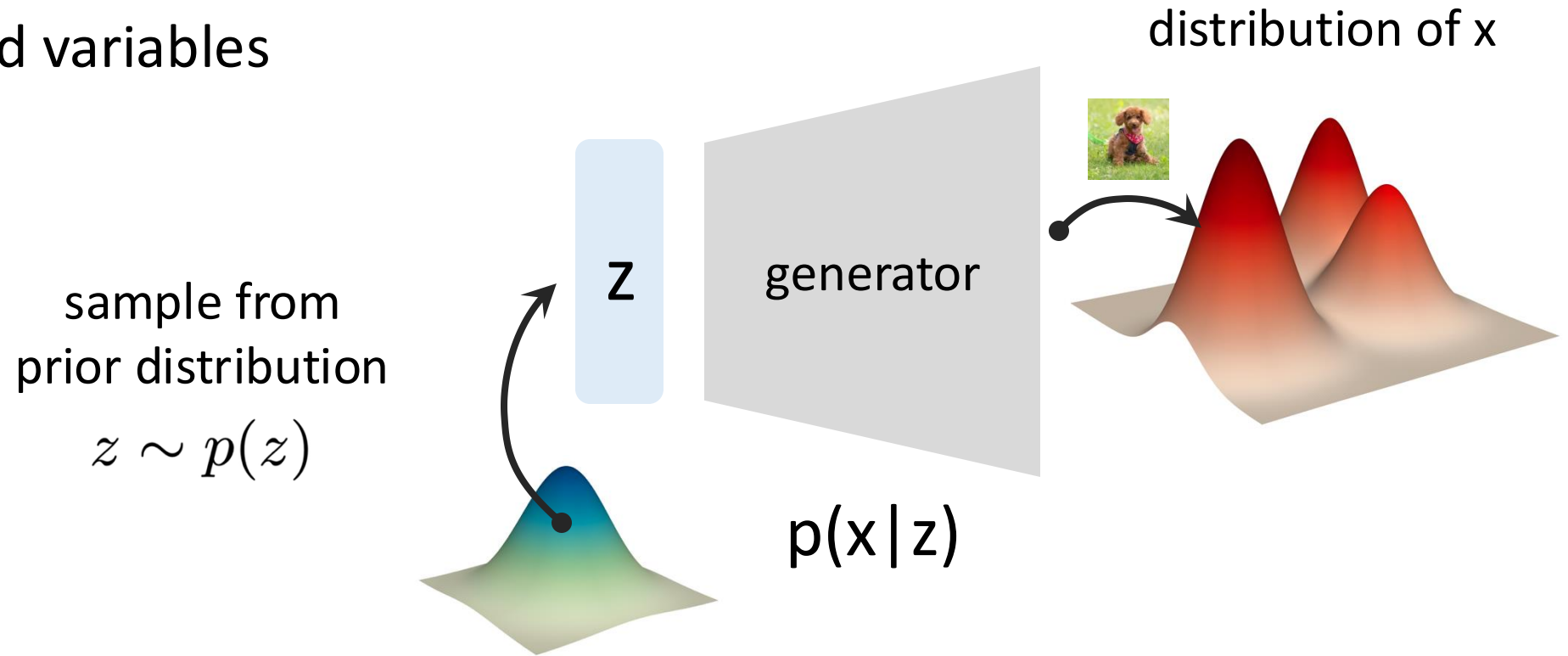
- $z$  - latent variables
- $x$  - observed variables



# Latent Variable Models

Assuming a data generation process:

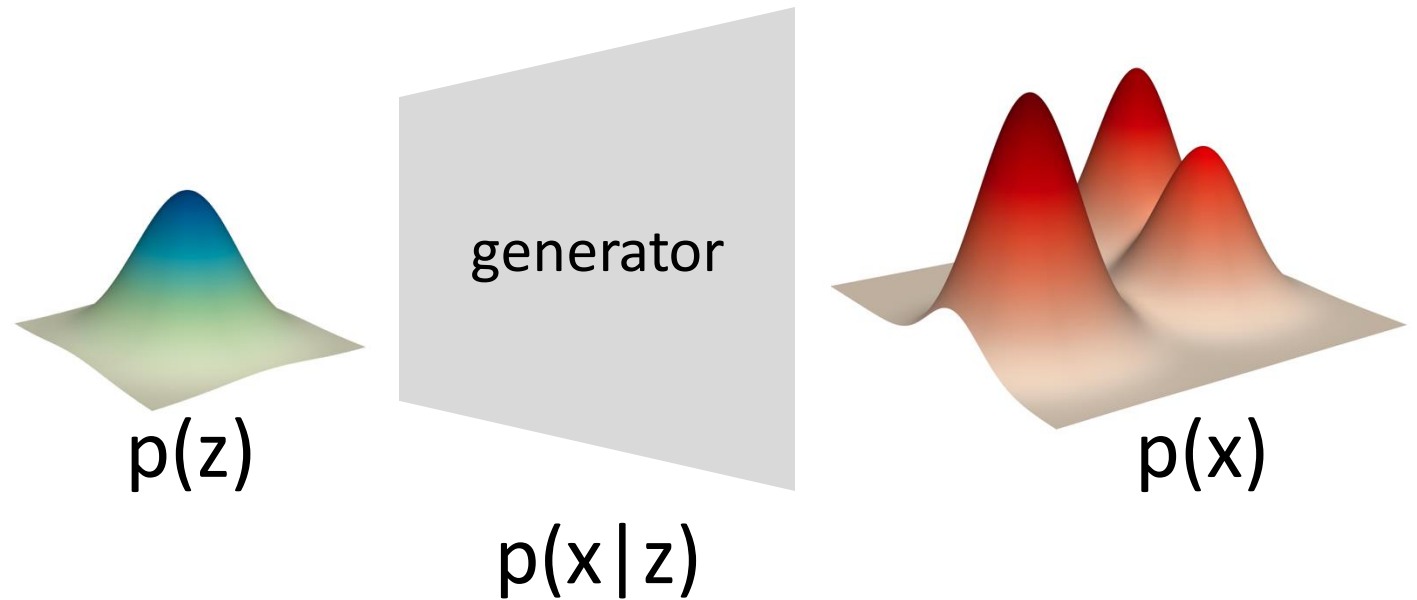
- $z$  - latent variables
- $x$  - observed variables



# Latent Variable Models

Assuming a data generation process:

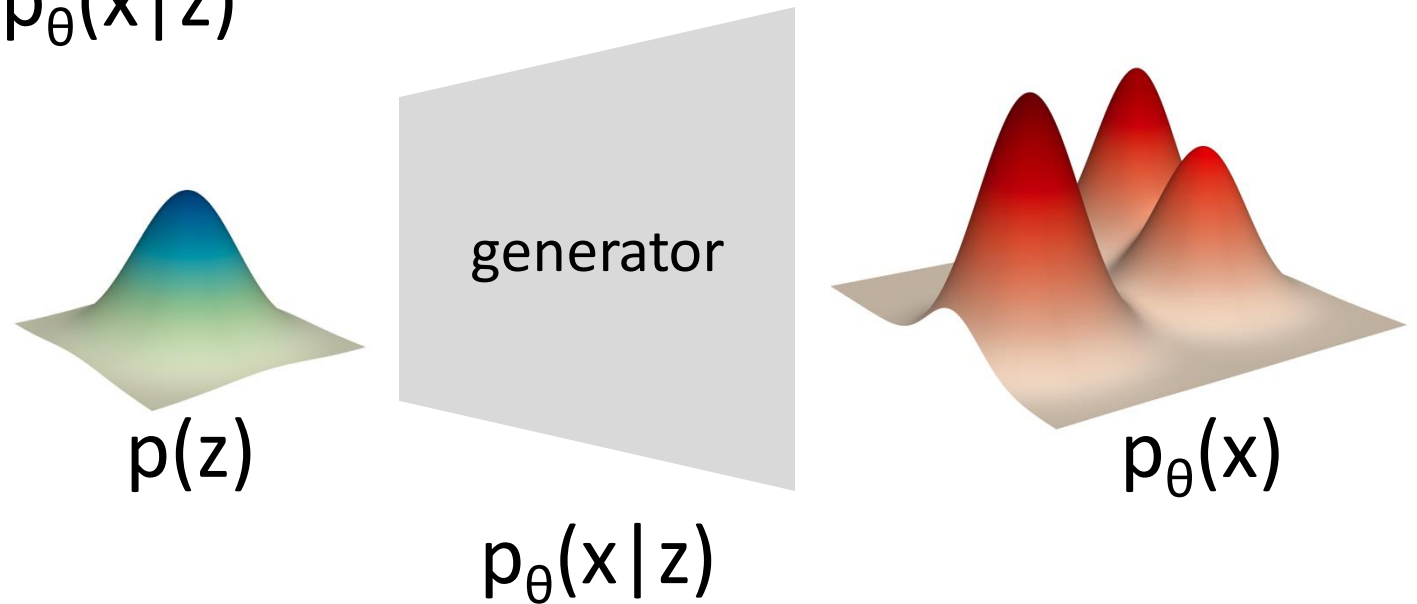
- $z$  - latent variables
- $x$  - observed variables



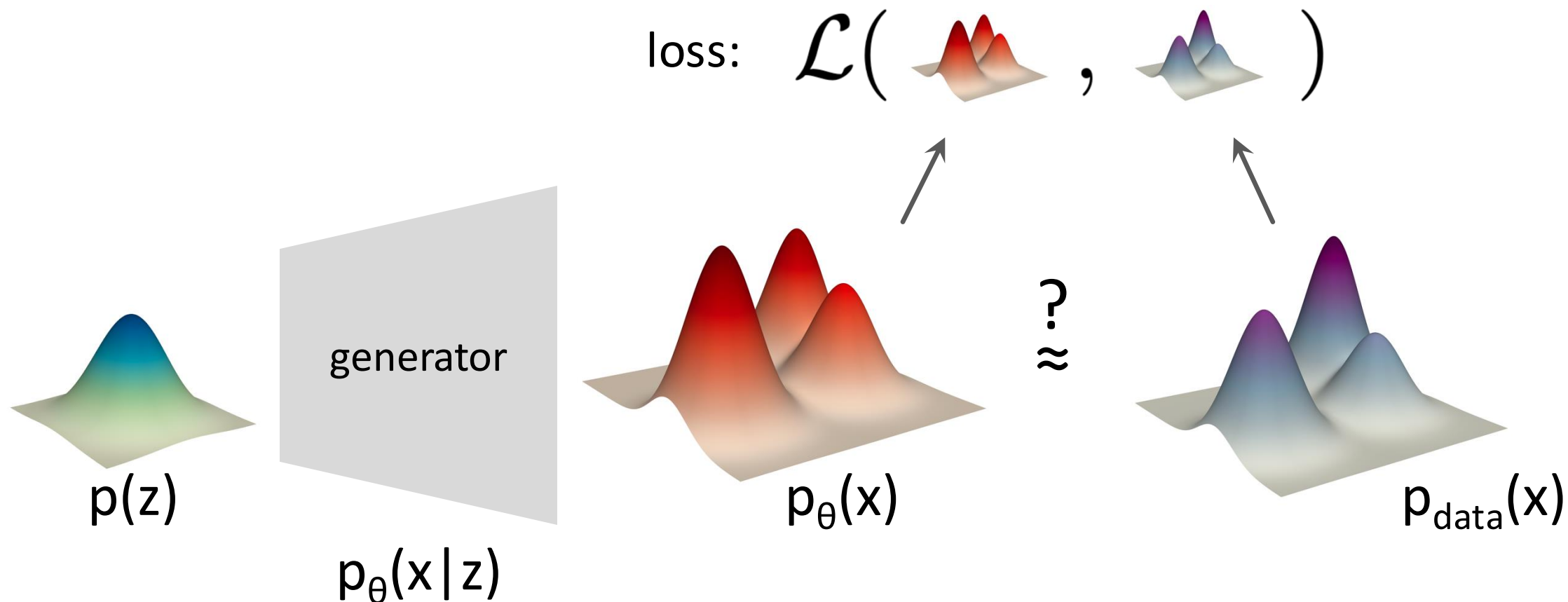
# Latent Variable Models

Represent a distribution by a neural network

- $\theta$  - learnable parameters
- represent a conditional:  $p_{\theta}(x|z)$

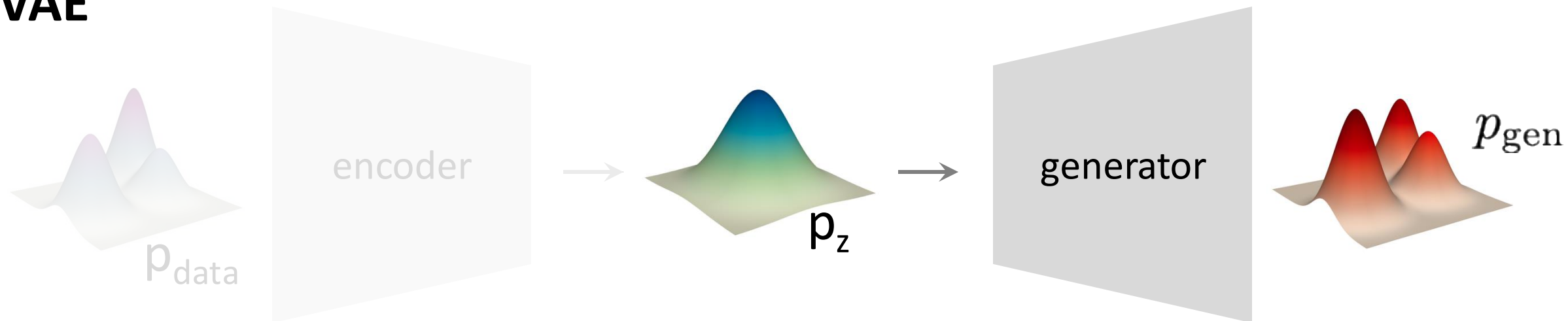


# Measuring How Good A Distribution Is

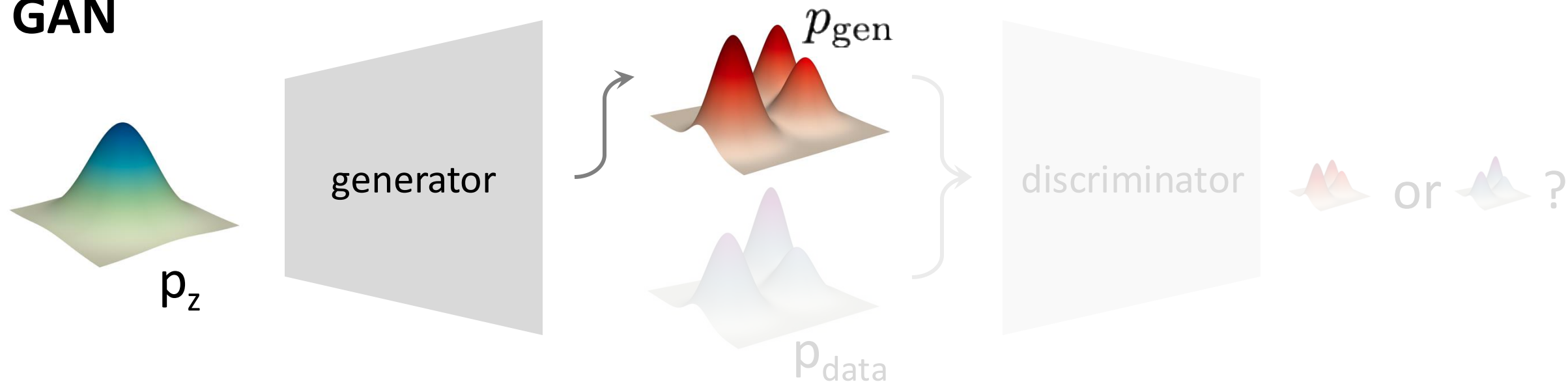


- A core topic in generative modeling: make it **differentiable**, **computable**, and **tractable**

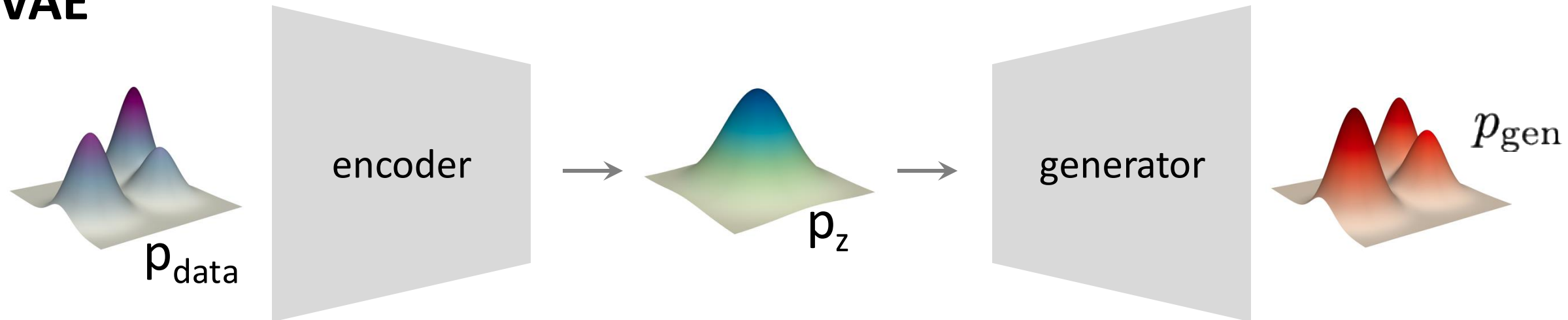
## VAE



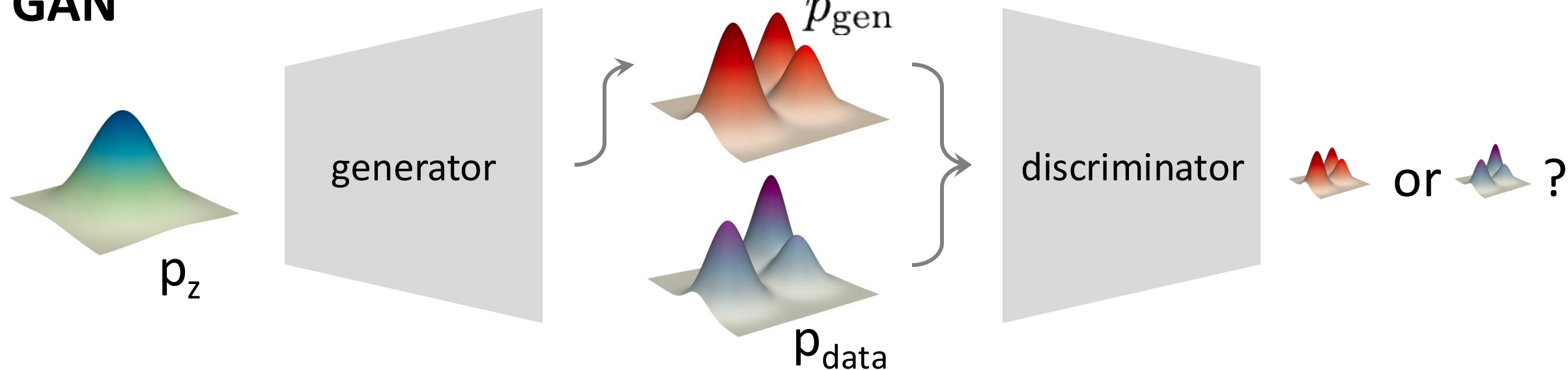
## GAN



## VAE



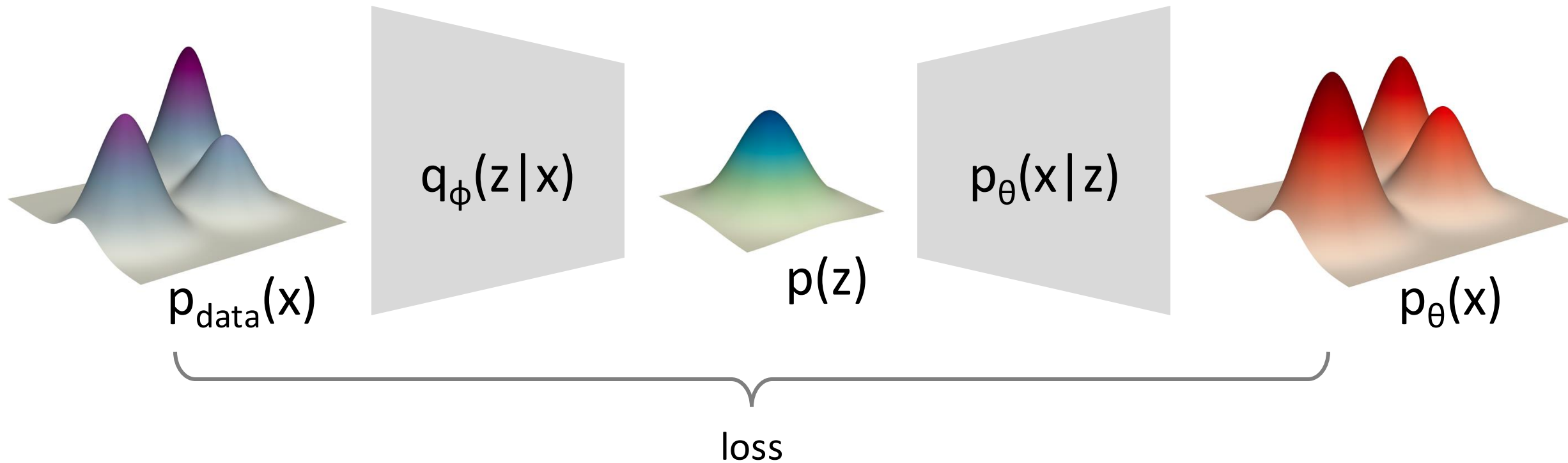
## GAN



# **Variational Autoencoder (VAE)**

# Variational Autoencoder: Overview

- **Encoder:** data to latent, parameterized by  $\phi$
- **Decoder:** latent to data, parameterized by  $\theta$



# Maximum Likelihood Estimation (MLE)

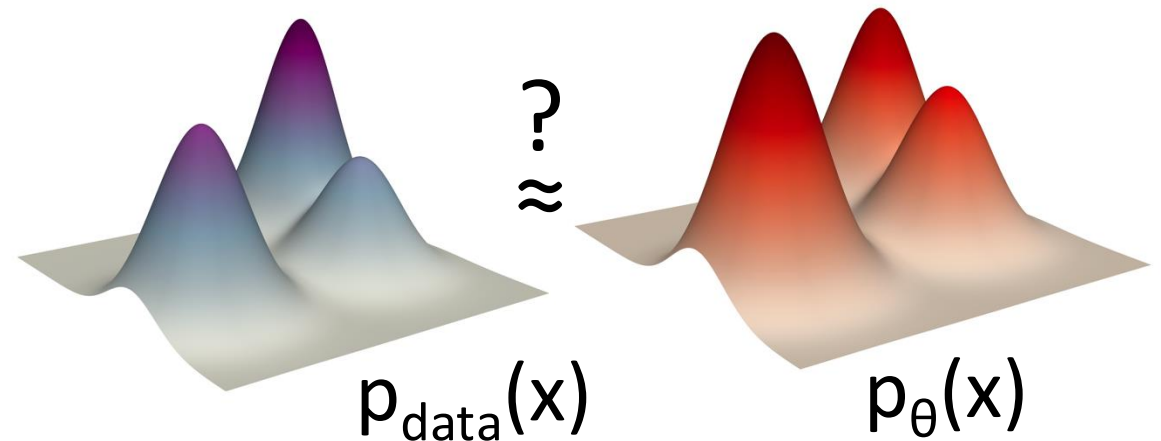
Minimize KL-Divergence:

$$\min_{\theta} \mathcal{D}_{\text{KL}}(p_{\text{data}} \parallel p_{\theta})$$

⇒ **Maximize** likelihood:

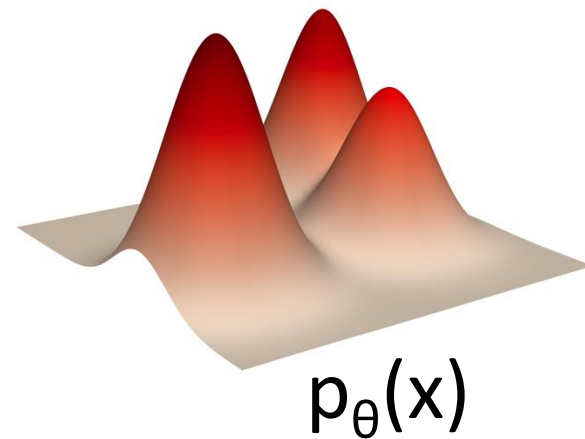
$$\max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\theta}(x)$$

$$\begin{aligned} & \arg \min_{\theta} \mathcal{D}_{\text{KL}}(p_{\text{data}} \parallel p_{\theta}) && \text{tl; dr} \\ = & \arg \min_{\theta} \sum_x p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_{\theta}(x)} \\ = & \arg \min_{\theta} \sum_x -p_{\text{data}}(x) \log p_{\theta}(x) + \text{const} \\ = & \arg \max_{\theta} \sum_x p_{\text{data}}(x) \log p_{\theta}(x) \\ = & \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\theta}(x) \end{aligned}$$



# Variational Autoencoder (VAE)

We want to maximize  $\mathbb{E}_{x \sim p_{data}} \log p_{\theta}(x)$

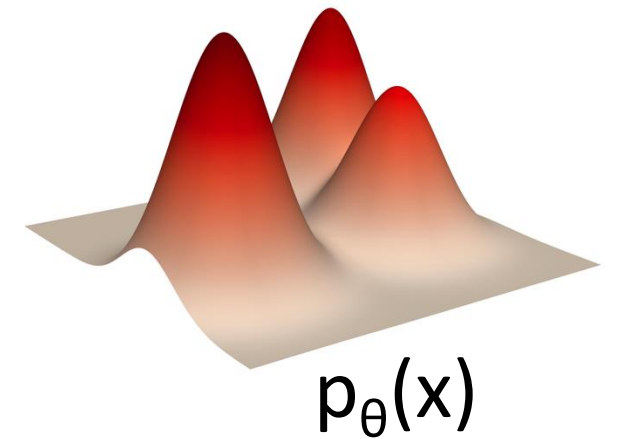
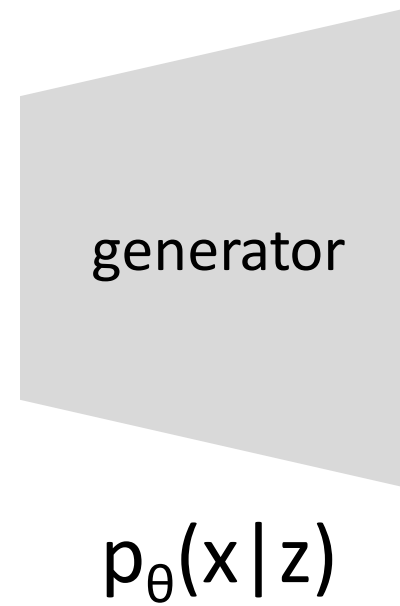
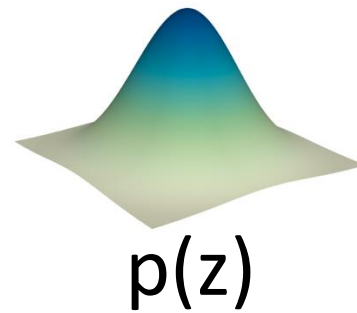


# Variational Autoencoder (VAE)

We want to maximize  $\mathbb{E}_{x \sim p_{data}} \log p_{\theta}(x)$

with  $p_{\theta}(x)$  represented as:

$$p_{\theta}(x) = \int_z p_{\theta}(x|z)p(z)dz$$



# Variational Autoencoder (VAE)

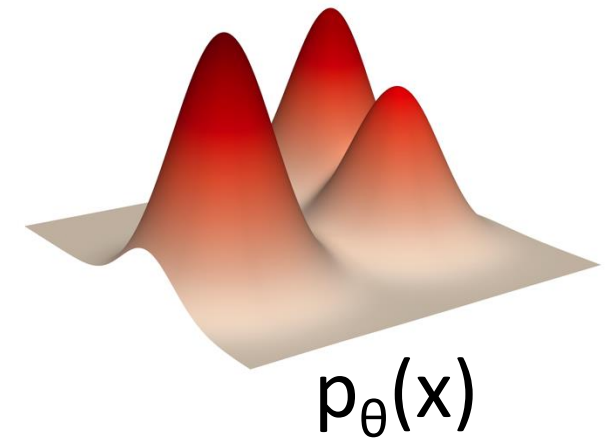
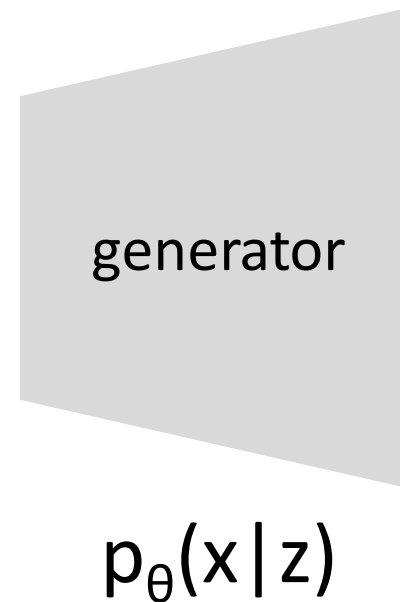
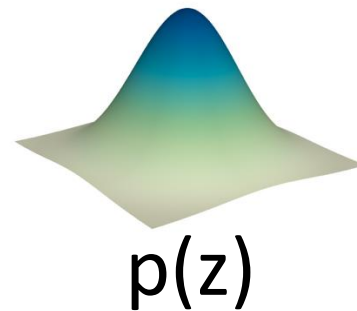
We want to maximize  $\mathbb{E}_{x \sim p_{data}} \log p_{\theta}(x)$

with  $p_{\theta}(x)$  represented as:

$$p_{\theta}(x) = \int_z \boxed{p_{\theta}(x|z)} \boxed{p(z)} dz$$

**Two** sets of unknowns:

- We need to optimize for  $\theta$
- We can't control “true”  $p(z)$



Idea: introduce a “controllable” distribution  $q_{\phi}(z)$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

$$\log p_{\theta}(x)$$

$$= \int_z q(z) \log p_{\theta}(x) dz$$

- valid for any distribution  $q(z)$

$$= \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz$$

- Bayes' rule

$$= \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz$$

$$= \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz$$

$$= \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right)$$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

$$\begin{aligned} & \log p_{\theta}(x) \\ = & \int_z q(z) \log p_{\theta}(x) dz \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz & \bullet \text{ valid for any distribution } q(z) \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz & \bullet \text{ Bayes' rule} \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz & \bullet \text{ just algebra} \\ = & \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz & \bullet \text{ just algebra} \\ = & \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right) \end{aligned}$$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

$$\begin{aligned} & \log p_{\theta}(x) \\ = & \int_z q(z) \log p_{\theta}(x) dz \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz & \begin{aligned} & \bullet \text{ valid for any distribution } q(z) \\ & \bullet \text{ Bayes' rule} \end{aligned} \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz & \bullet \text{ just algebra} \\ = & \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz & \bullet \text{ just algebra} \\ = & \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right) \end{aligned}$$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

$$\begin{aligned} & \log p_{\theta}(x) \\ = & \int_z q(z) \log p_{\theta}(x) dz \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz & \begin{aligned} & \bullet \text{ valid for any distribution } q(z) \\ & \bullet \text{ Bayes' rule} \end{aligned} \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz & \bullet \text{ just algebra} \\ = & \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz & \bullet \text{ just algebra} \\ = & \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right) \end{aligned}$$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

$$\begin{aligned} & \log p_{\theta}(x) \\ = & \int_z q(z) \log p_{\theta}(x) dz \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz & \begin{aligned} & \bullet \text{ valid for any distribution } q(z) \\ & \bullet \text{ Bayes' rule} \end{aligned} \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz & \bullet \text{ just algebra} \\ = & \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz & \bullet \text{ just algebra} \\ = & \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right) \end{aligned}$$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

$$\begin{aligned} & \log p_{\theta}(x) \\ = & \int_z q(z) \log p_{\theta}(x) dz \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz & \begin{aligned} & \bullet \text{ valid for any distribution } q(z) \\ & \bullet \text{ Bayes' rule} \end{aligned} \\ = & \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz & \bullet \text{ just algebra} \\ = & \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz & \bullet \text{ just algebra} \\ = & \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right) \end{aligned}$$

# Evidence Lower Bound (ELBO)

- Rewrite log-likelihood using latent  $z$

intractable

$$\log p_{\theta}(x)$$

$$= \int_z q(z) \log p_{\theta}(x) dz$$

- valid for any distribution  $q(z)$

$$= \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \right) dz$$

- Bayes' rule

$$= \int_z q(z) \log \left( \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(z|x)} \frac{q(z)}{q(z)} \right) dz$$

$$= \int_z q(z) \left( \log p_{\theta}(x|z) + \log \frac{p_{\theta}(z)}{q(z)} + \log \frac{q(z)}{p_{\theta}(z|x)} \right) dz$$

$$= \mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right] - \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z) \right) + \mathcal{D}_{\text{KL}} \left( q(z) || p_{\theta}(z|x) \right)$$

tractable

tractable

intractable

# Evidence Lower Bound (ELBO)

$$\text{intractable} \quad \boxed{\log p_{\theta}(x)} - \boxed{\mathcal{D}_{\text{KL}}(q(z) || p_{\theta}(z|x))} \quad \text{intractable}$$

$$= \underbrace{\mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right]}_{\text{tractable}} - \underbrace{\mathcal{D}_{\text{KL}}(q(z) || p_{\theta}(z))}_{\text{tractable}}$$

# Evidence Lower Bound (ELBO)

$$\begin{aligned} & \text{intractable} \quad \boxed{\log p_{\theta}(x)} - \boxed{\mathcal{D}_{\text{KL}}(q(z) || p_{\theta}(z|x))} \quad \text{intractable} \\ &= \underbrace{\boxed{\mathbb{E}_{z \sim q(z)} [\log p_{\theta}(x|z)]}}_{\text{tractable}} - \underbrace{\boxed{\mathcal{D}_{\text{KL}}(q(z) || p_{\theta}(z))}}_{\text{tractable}} \\ & \hspace{15em} \text{ELBO} \end{aligned}$$

**ELBO:**

- It's lower bound of  $\log p_{\theta}(x)$
- This formulation holds for any distribution  $q(z)$

# Evidence Lower Bound (ELBO)

$$\underbrace{\mathbb{E}_{z \sim \cancel{q(z)}}^{\cancel{q_\phi(z|x)}} \left[ \log p_\theta(x|z) \right] - \mathcal{D}_{\text{KL}} \left( \cancel{q(z)} \parallel p_\theta(z) \right)}_{\text{ELBO}}$$

The diagram shows the ELBO formula with two terms enclosed in green boxes. The first term is  $\mathbb{E}_{z \sim \cancel{q(z)}}^{\cancel{q_\phi(z|x)}} [\log p_\theta(x|z)]$  and the second term is  $-\mathcal{D}_{\text{KL}}(\cancel{q(z)} \parallel p_\theta(z))$ . Both terms are labeled "tractable" in green. A bracket underneath both terms is labeled "ELBO".

## ELBO:

- It's lower bound of  $\log p_\theta(x)$
- This formulation holds for any distribution  $q(z)$  , so ...
- we can parameterize  $q(z)$  by  $q_\phi(z|x)$

# Evidence Lower Bound (ELBO)

$$\underbrace{\mathbb{E}_{z \sim q(z)} \left[ \log p_{\theta}(x|z) \right]}_{\text{tractable}} - \underbrace{\mathcal{D}_{\text{KL}} \left( q(z) \parallel p_{\theta}(z) \right)}_{\text{tractable}}$$

ELBO

**\*Note:** ELBO is not specific to VAE.

## ELBO:

- It's lower bound of  $\log p_{\theta}(x)$
- This formulation holds for any distribution  $q(z)$  , so ...
- we can parameterize  $q(z)$  by  $q_{\phi}(z|x)$
- and let  $p_{\theta}(z)$  be a fixed, known prior  $p(z)$  (e.g., Gaussian)

# Variational Autoencoder

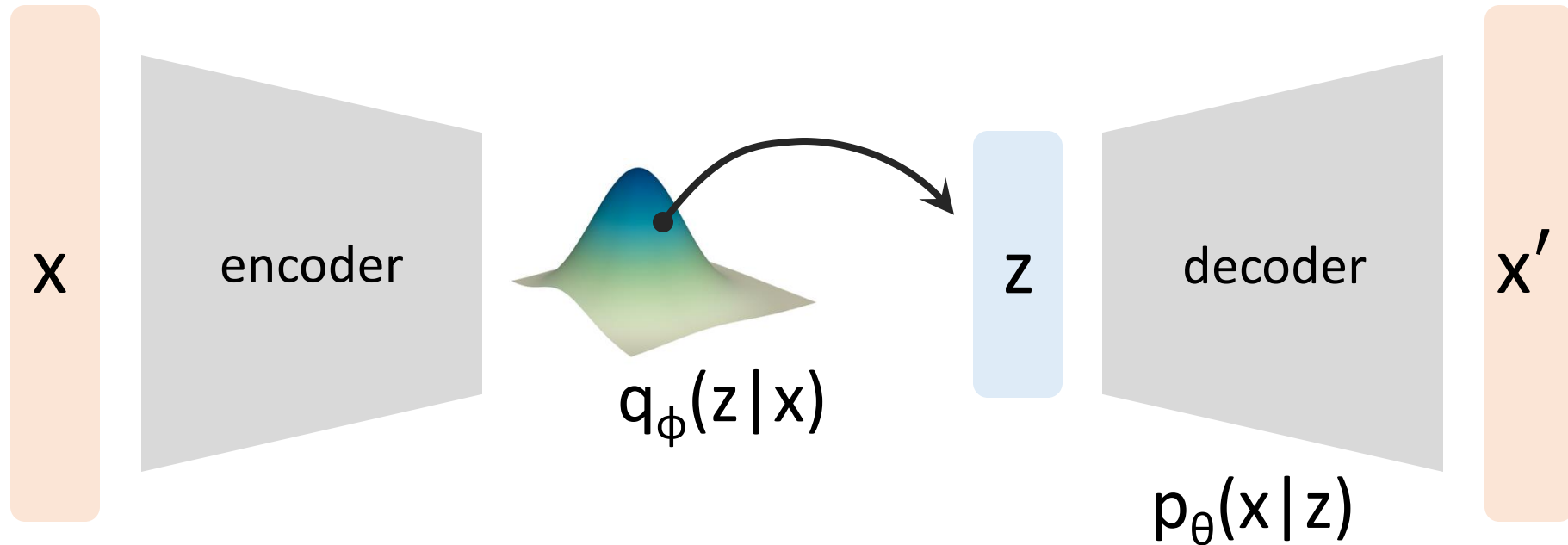
Maximize ELBO  $\Rightarrow$  minimize:

$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$

# Variational Autoencoder

Maximize ELBO  $\Rightarrow$  minimize:

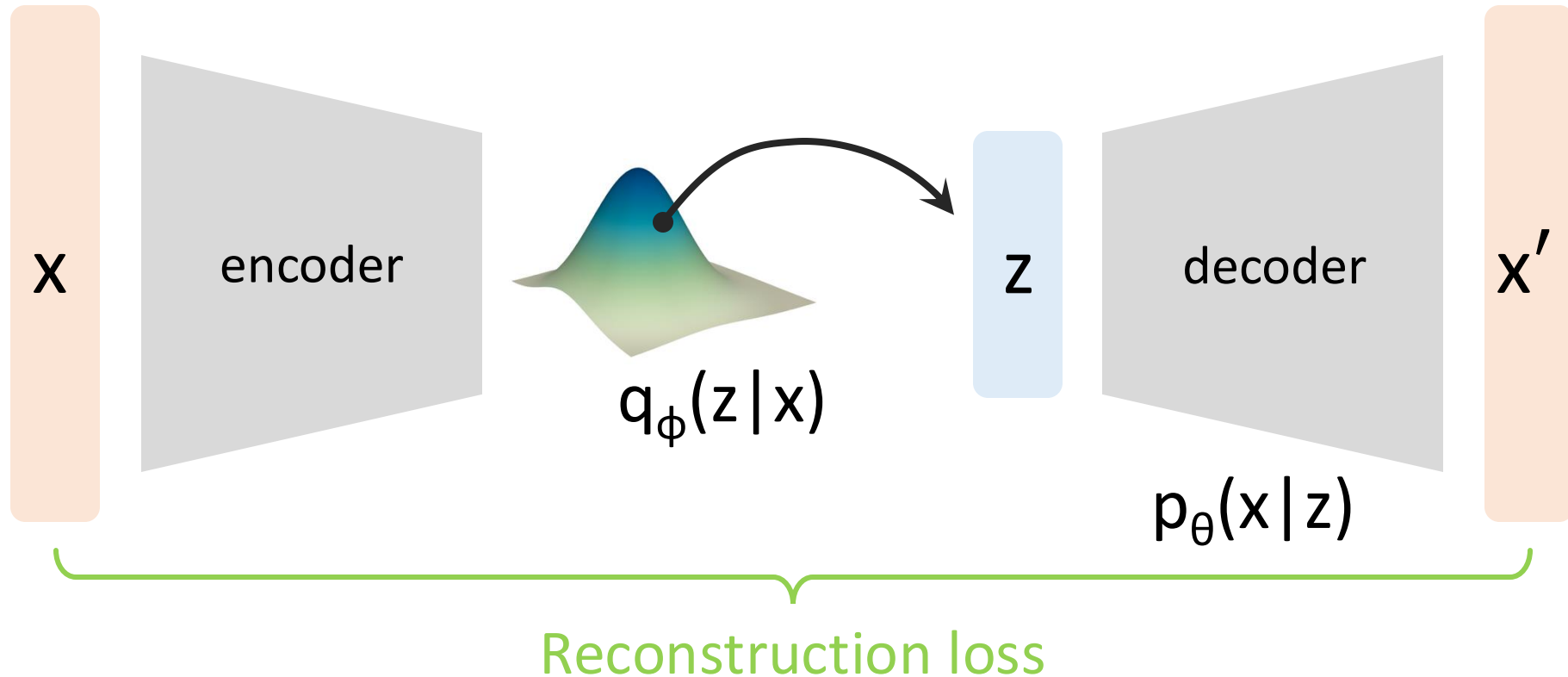
$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$



# Variational Autoencoder

Maximize ELBO  $\Rightarrow$  minimize:

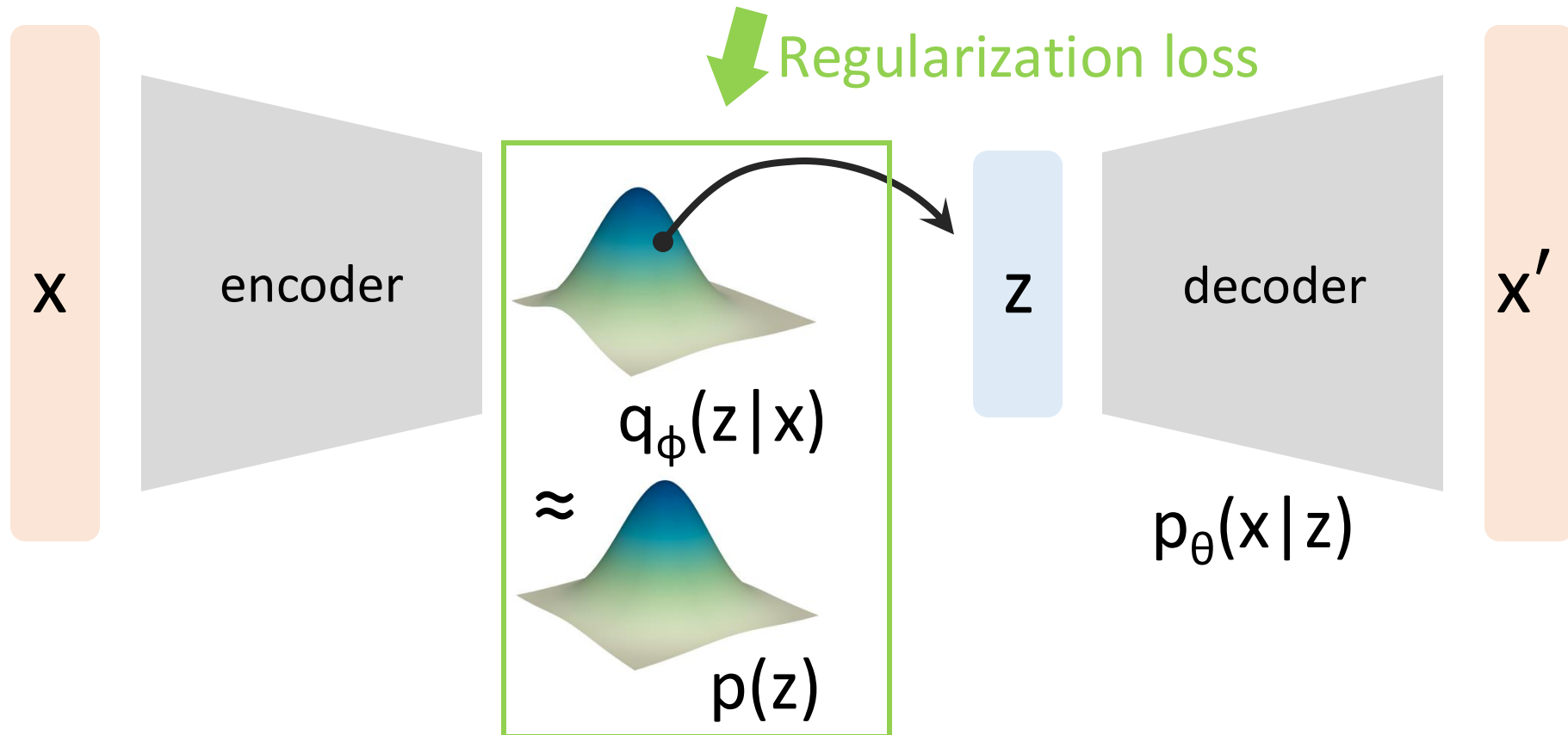
$$\mathcal{L}_{\theta, \phi}(x) = \boxed{-\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right]} + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$



# Variational Autoencoder

Maximize ELBO  $\Rightarrow$  minimize:

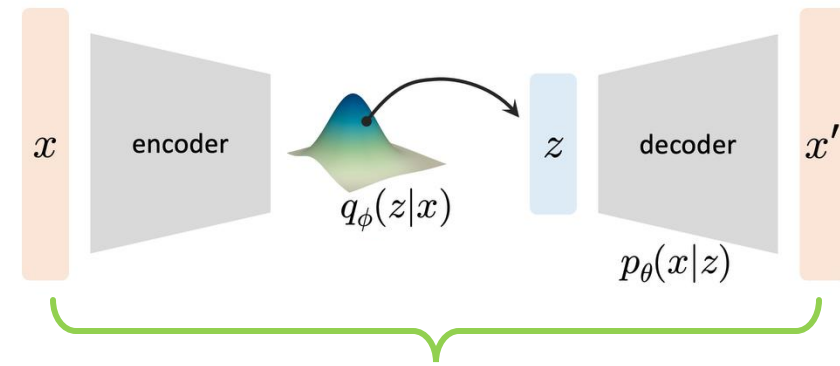
$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$



# Variational Autoencoder

## Reconstruction loss

$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}}(q_{\phi}(z|x) || p(z))$$

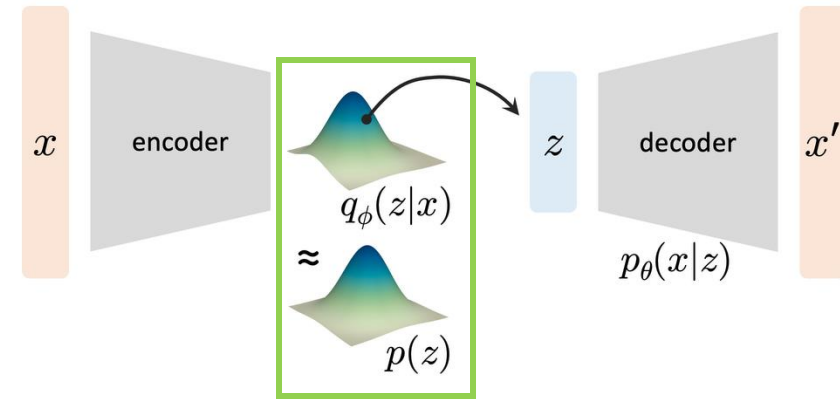


Example: L2 loss

- one-step Monte Carlo:  $z \sim q_{\phi}(z|x)$
- map  $z$  by decoder net:  $g_{\theta}(z) \rightarrow x'$  network estimates distribution's parameters (Gaussian's mean)
- model  $p_{\theta}(x|z)$  by Gaussian:  $p_{\theta}(x|z) = \mathcal{N}(x | x', \sigma_0^2)$  (assume fixed std)
- negative log likelihood:  $\frac{1}{2\sigma_0^2} \|x - x'\|^2 + \text{const}$
- L2 loss  $\Rightarrow$  a Gaussian neighborhood around data point  $x$

# Variational Autoencoder

Regularization loss



$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] + \mathcal{D}_{\text{KL}}(q_\phi(z|x) || p(z))$$

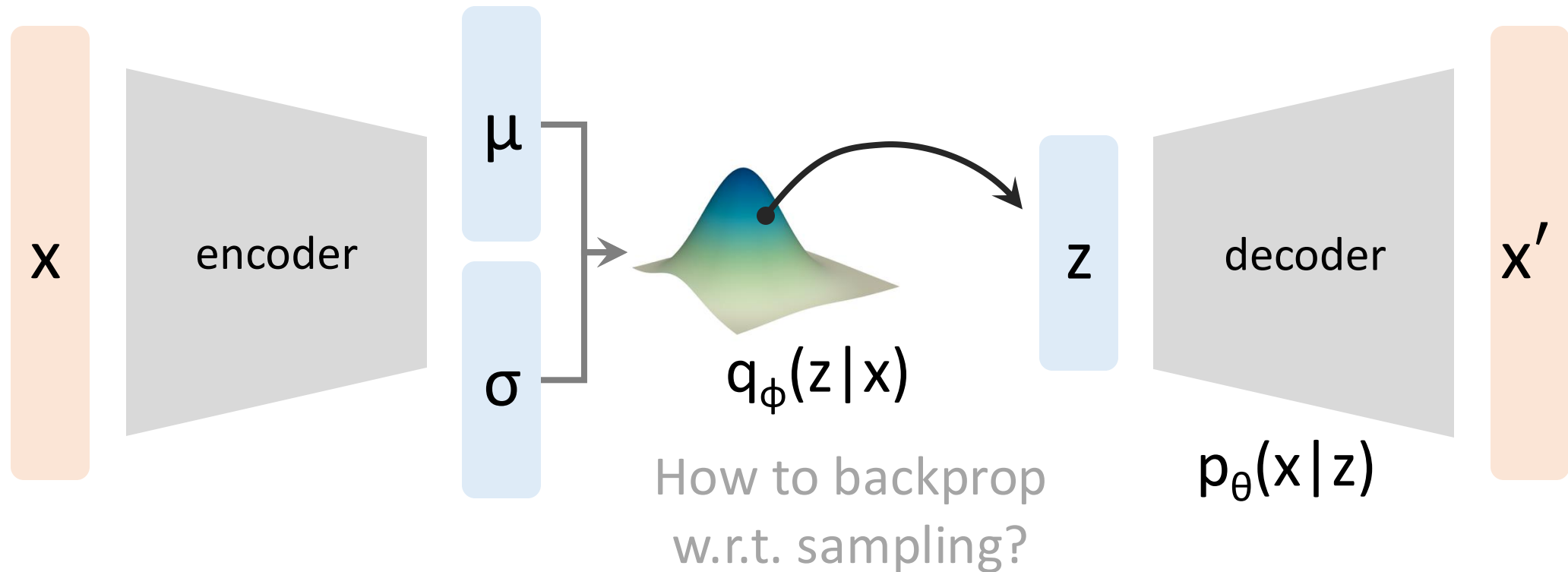
Example: Gaussian prior

- let  $p(z) = \mathcal{N}(z | 0, \mathbf{I})$
- model  $q_\phi(z|x)$  by Gaussian:  $\mathcal{N}(z | \mu, \sigma)$
- map  $x$  by encoder net:  $f_\phi(x) \rightarrow \mu, \sigma$  again, network estimates distribution's parameters
- compute loss analytically:  $\mathcal{D}_{\text{KL}}(\mathcal{N}(z | \mu, \sigma) || \mathcal{N}(z | 0, \mathbf{I}))$
- fixed covariance  $\Rightarrow$  L2 loss on  $\mu$

# Variational Autoencoder

Maximize ELBO  $\Rightarrow$  minimize:

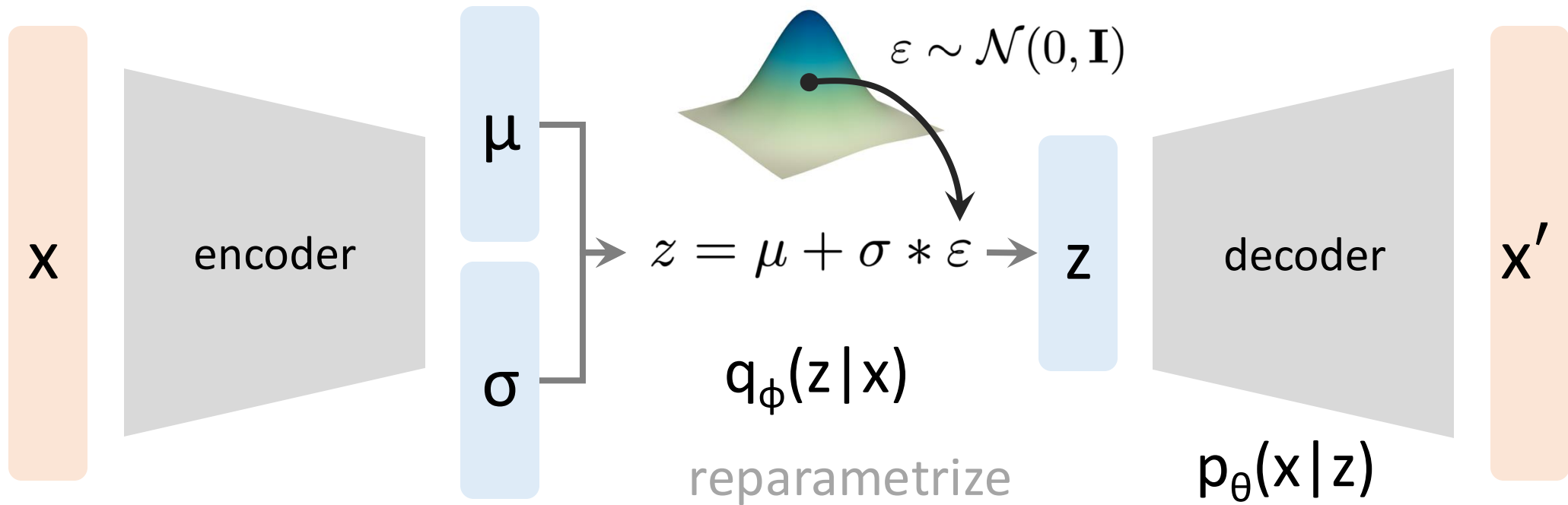
$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$



# Variational Autoencoder

Maximize ELBO  $\Rightarrow$  minimize:

$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$



# Variational Autoencoder

... so far, we have discussed an objective on **one**  $x$ :

$$\mathcal{L}_{\theta, \phi}(x) = -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right)$$

Overall loss is expectation over data:

$$\mathcal{L}_{\theta, \phi} = \mathbb{E}_{x \sim p_{data}(x)} \left[ -\mathbb{E}_{z \sim q_{\phi}(z|x)} \left[ \log p_{\theta}(x|z) \right] + \mathcal{D}_{\text{KL}} \left( q_{\phi}(z|x) || p(z) \right) \right]$$

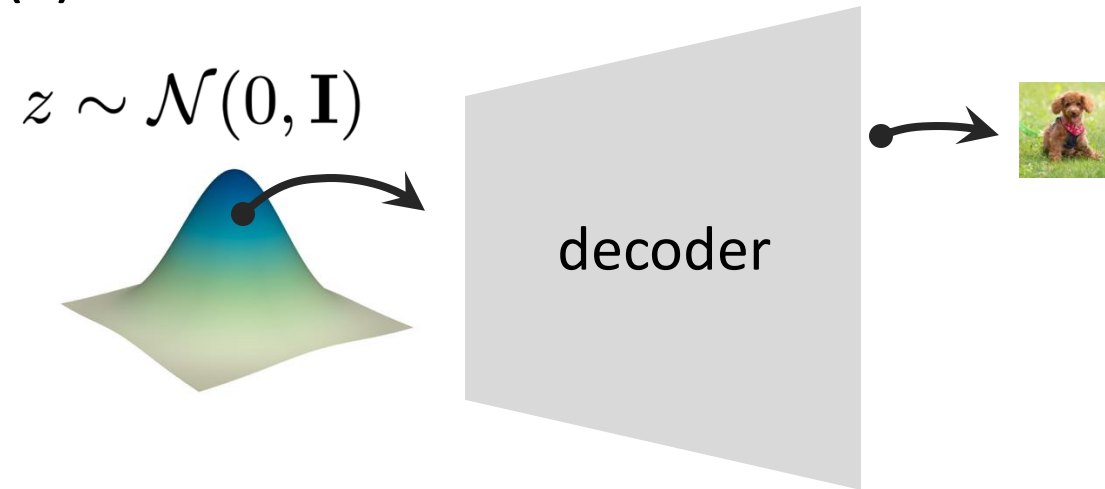
The formulation of “**x-conditional**” then “**marginalization**” is a common strategy

- Diffusion, Flow Matching, ...

# Variational Autoencoder

Inference (generation):

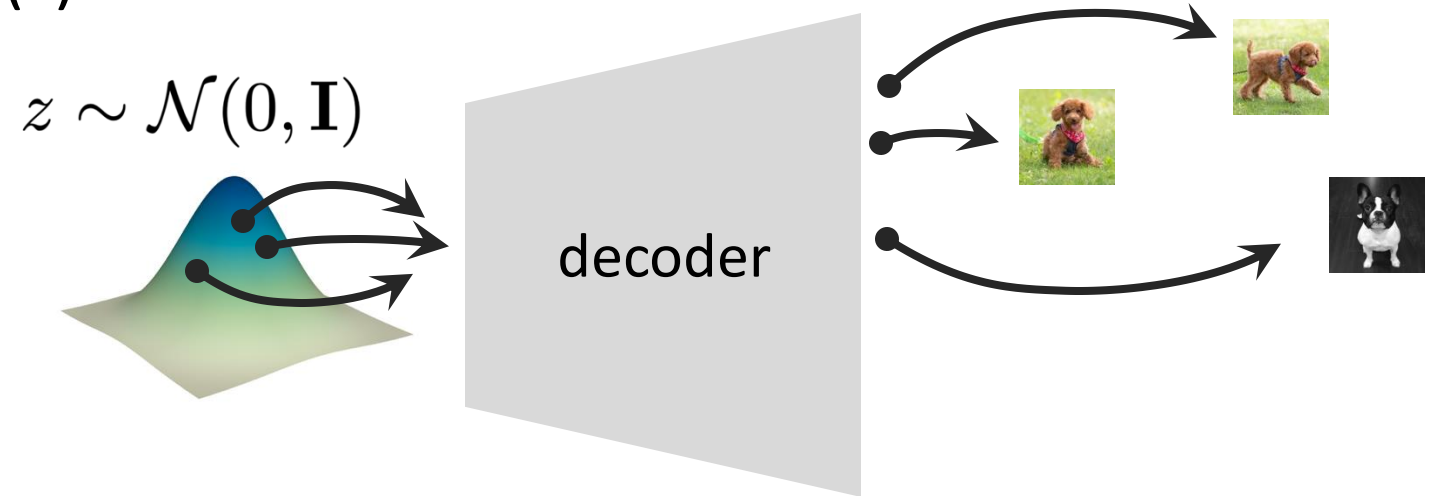
- sample  $z$  from:  $\mathcal{N}(0, \mathbf{I})$
- map  $z$  by decoder net:  $g_{\theta}(z)$



# Variational Autoencoder

Inference (generation):

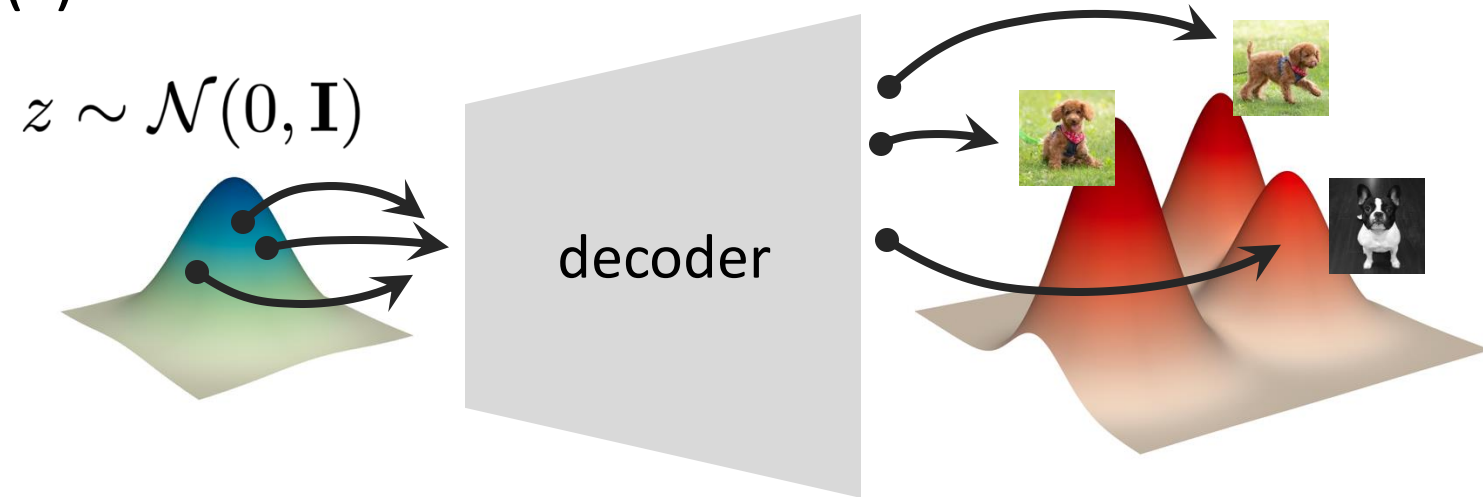
- sample  $z$  from:  $\mathcal{N}(0, \mathbf{I})$
- map  $z$  by decoder net:  $g_{\theta}(z)$



# Variational Autoencoder

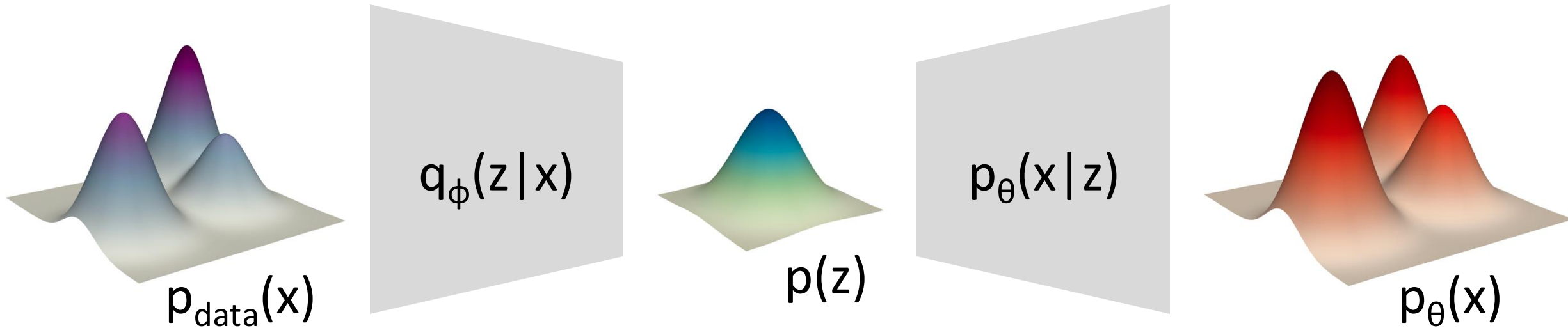
Inference (generation):

- sample  $z$  from:  $\mathcal{N}(0, \mathbf{I})$
- map  $z$  by decoder net:  $g_{\theta}(z)$



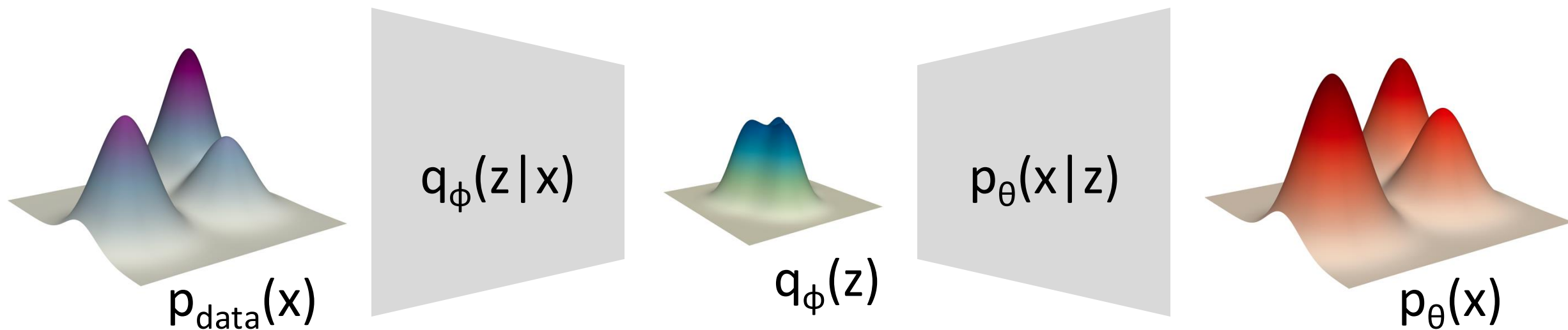
# A view of “Autoencoding Distributions”

- **Encoder:** data to latent
- **Decoder:** latent to data



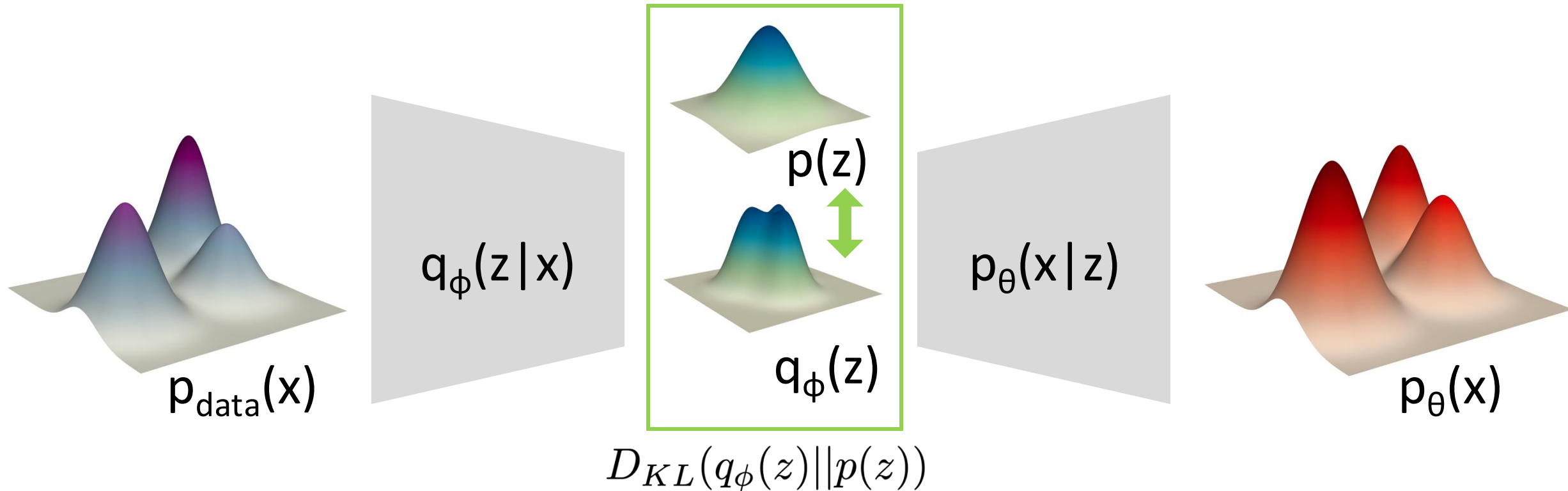
# A view of “Autoencoding Distributions”

- encoded latent distribution:  $q_{\phi}(z) = \int_x q_{\phi}(z|x)p_{data}(x)dx$



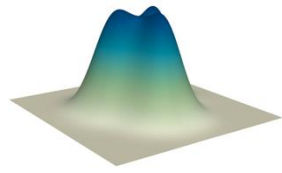
# A view of “Autoencoding Distributions”

- encoded latent distribution:  $q_{\phi}(z) = \int_x q_{\phi}(z|x)p_{data}(x)dx$

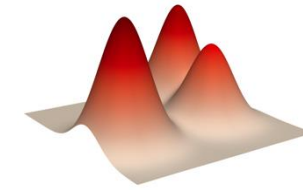
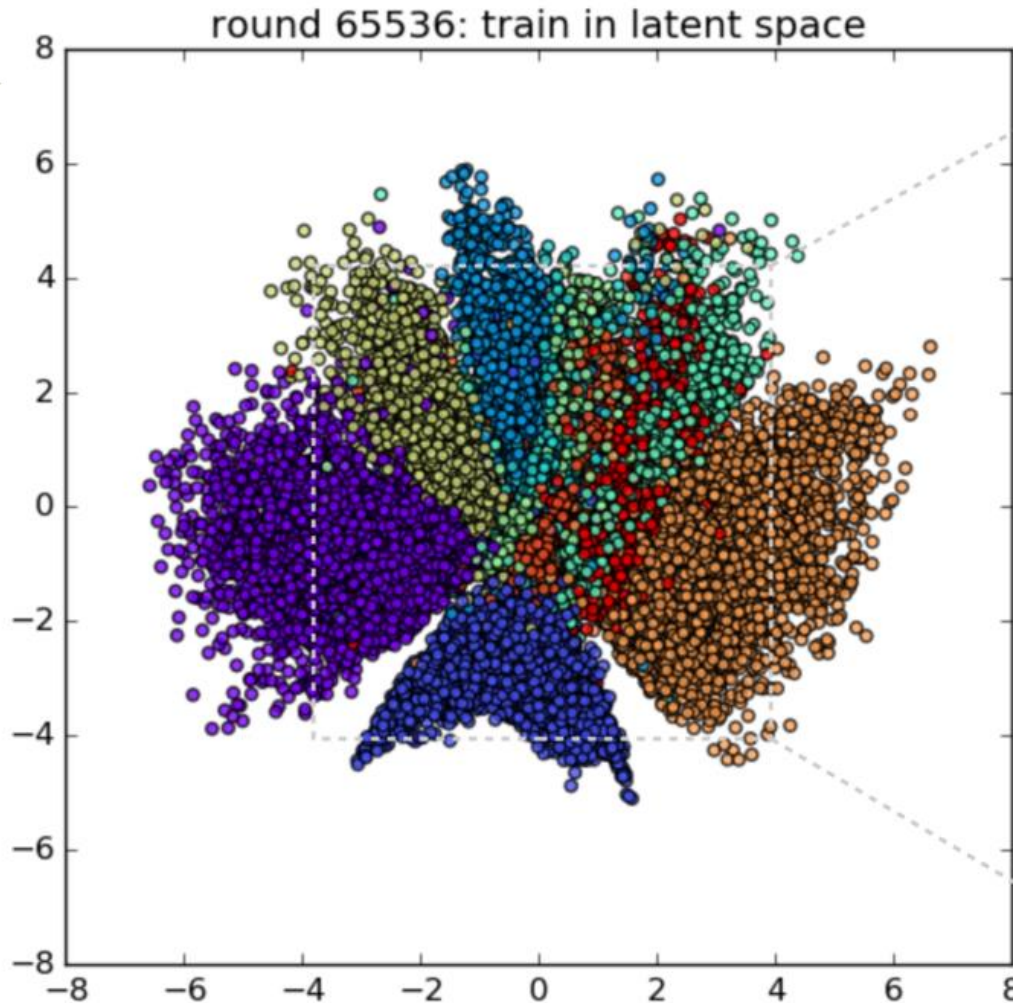


E.g., see “*InfoVAE: Information Maximizing Variational Autoencoders*”, 2017

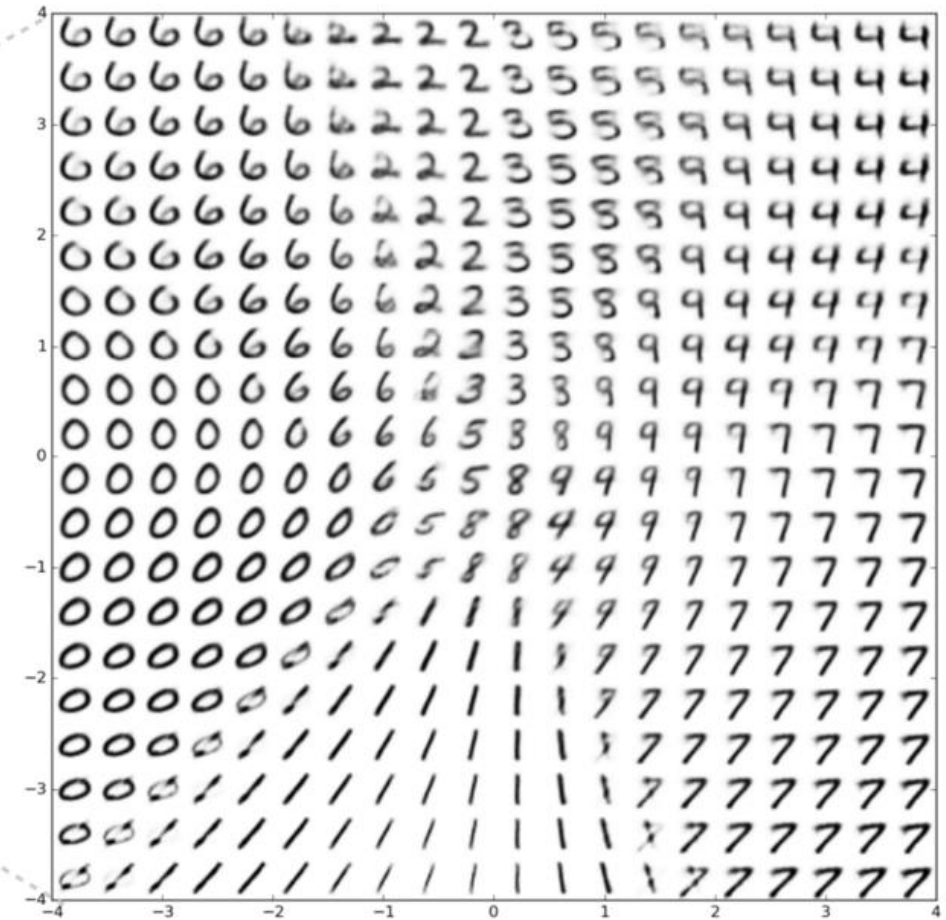
# VAE: 2D latent space on MNIST



$$q_{\phi}(z)$$



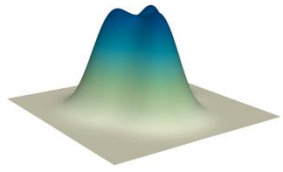
$$p_{\theta}(x)$$



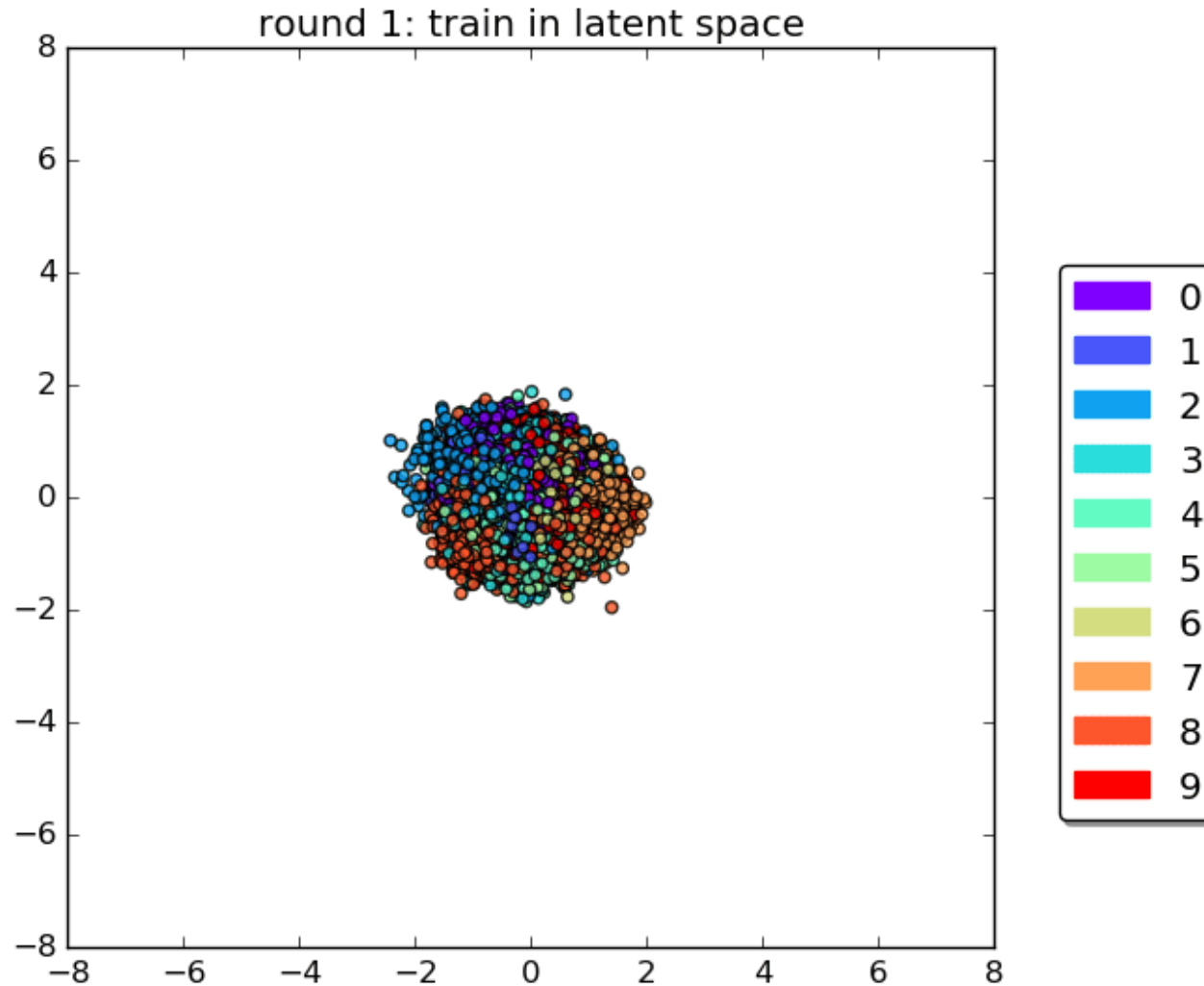
“Introducing Variational Autoencoders (in Prose and Code)”

<https://blog.fastforwardlabs.com/2016/08/12/introducing-variational-autoencoders-in-prose-and-code.html>

# VAE: 2D latent space on MNIST



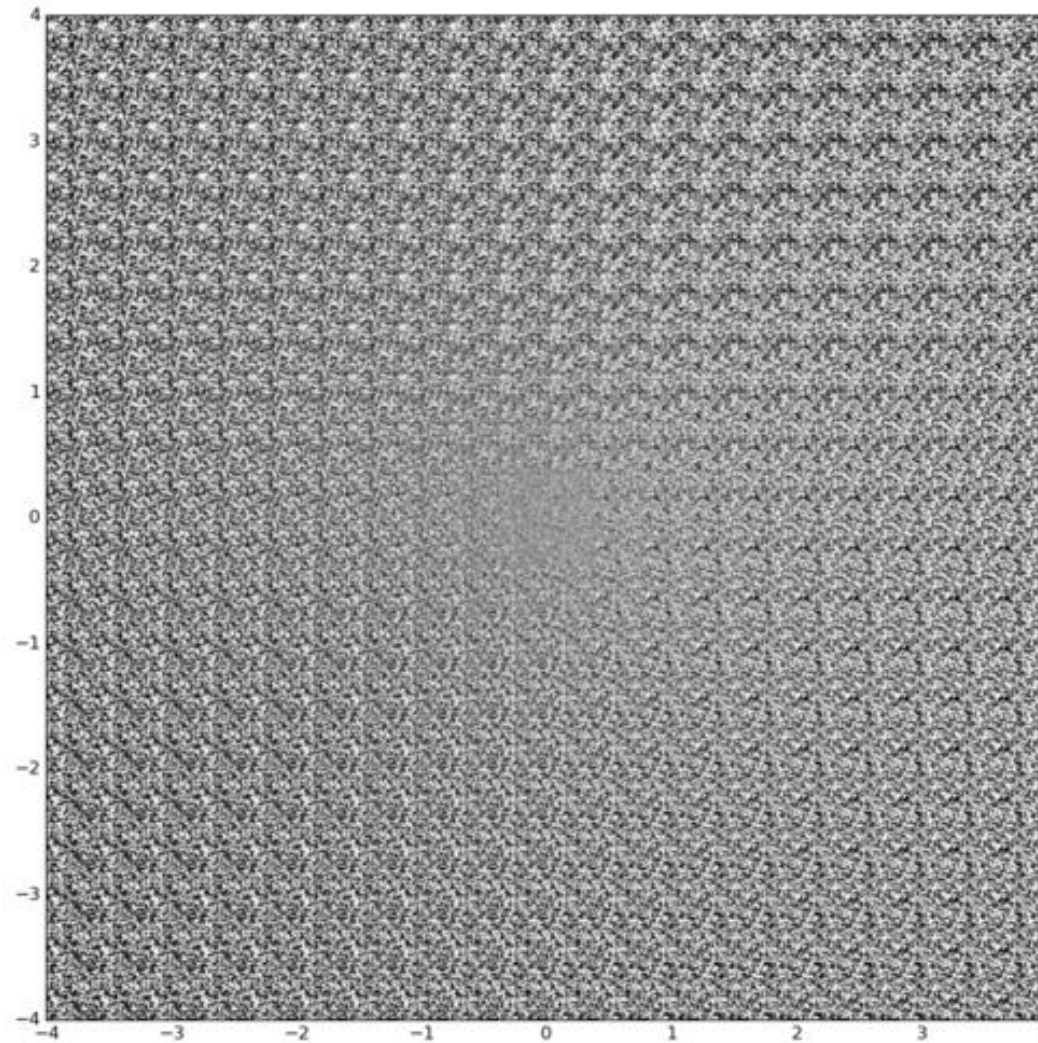
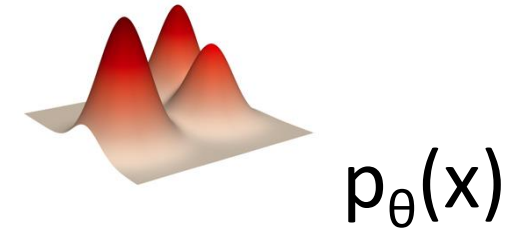
$q_\phi(z)$



To pdf users: this is animation. Check it on: “Introducing Variational Autoencoders (in Prose and Code)”

<https://blog.fastforwardlabs.com/2016/08/12/introducing-variational-autoencoders-in-prose-and-code.html>

# VAE: 2D latent space on MNIST



To pdf users: this is animation. Check it on: “Introducing Variational Autoencoders (in Prose and Code)”  
<https://blog.fastforwardlabs.com/2016/08/12/introducing-variational-autoencoders-in-prose-and-code.html>

# VAE results on 784-d MNIST data



(a) 2-D latent space

(b) 5-D latent space

(c) 10-D latent space

(d) 20-D latent space

Too strict to model the 784-d (28x28) joint distribution by independent distributions

# VAE: 2D latent space on “Frey Face” dataset



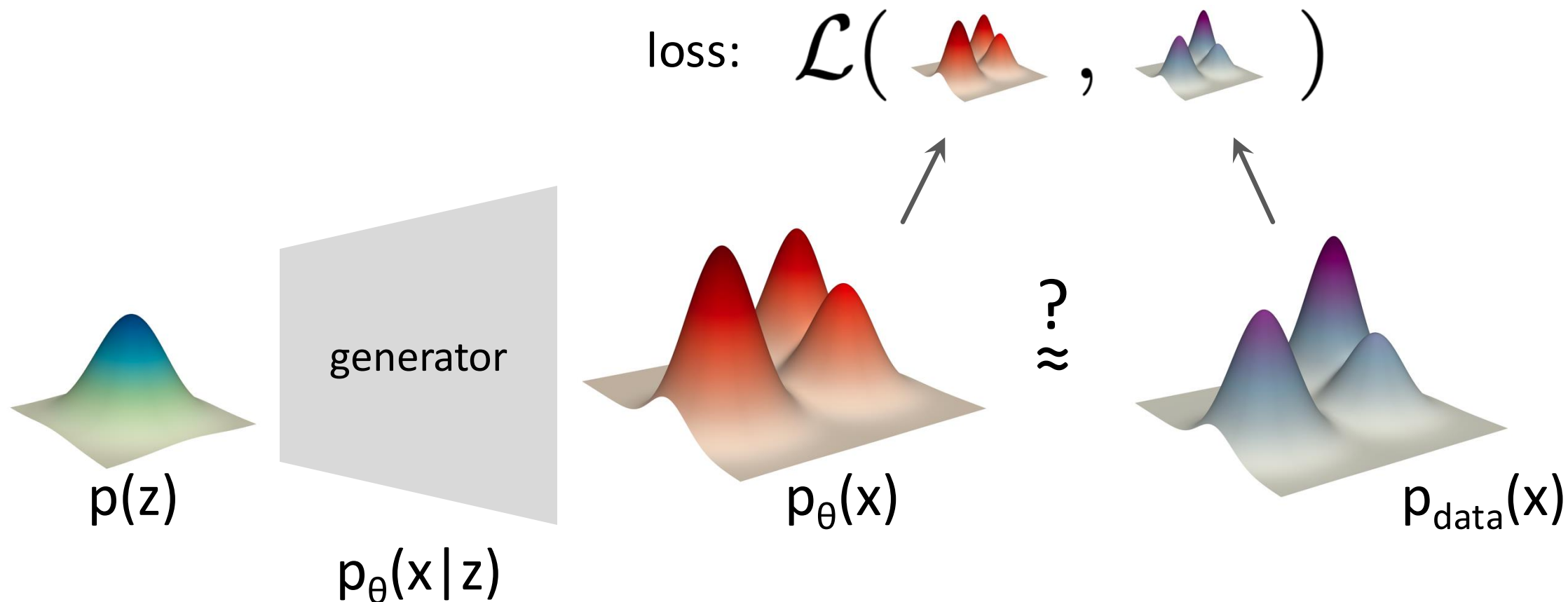
# Issues of VAE

- Gaussian prior is too **strict** an assumption
- Reconstruction loss assumes **independence** across output elements
- Difficult to model **high-dimensional, complex** distributions

**In practice** - VAE is often used as a module (tokenizer), and the prior is learned by other models (AR or Diffusion).

# **Generative Adversarial Network (GAN)**

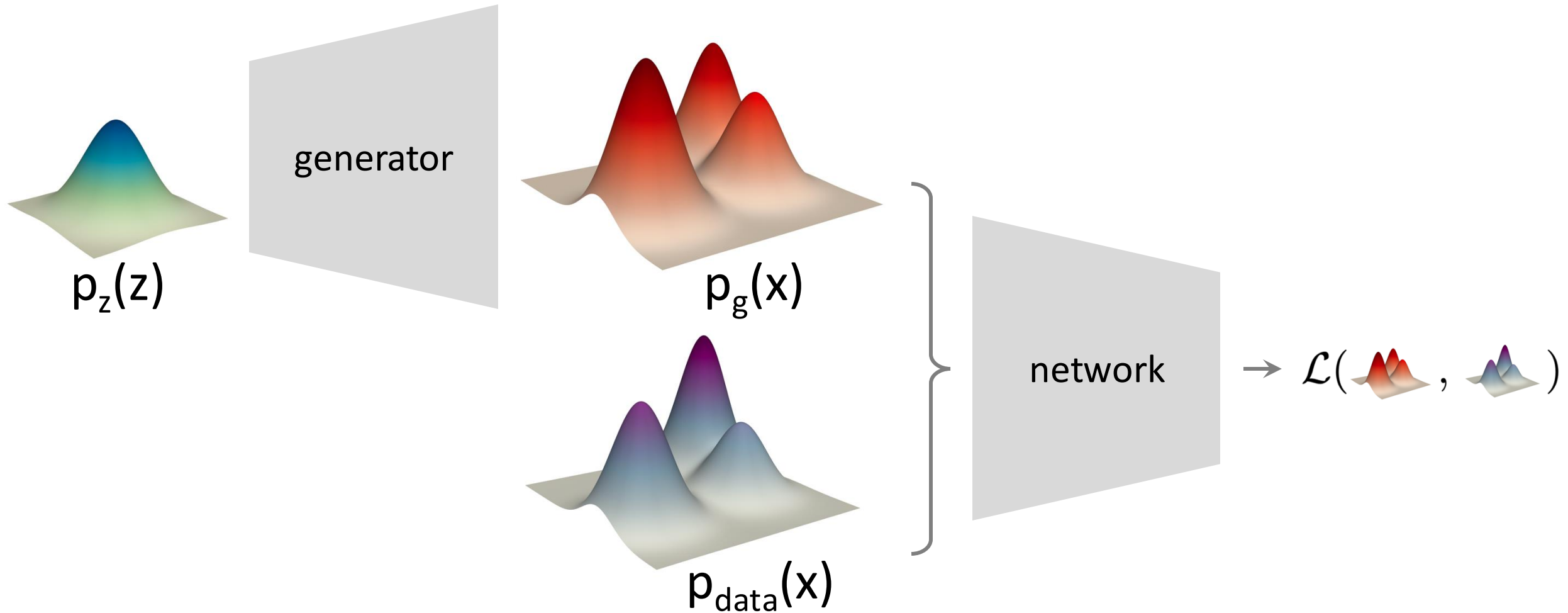
# [Recap]: Measuring How Good A Distribution Is



- A core topic in generative modeling: make it **differentiable**, **computable**, and **tractable**

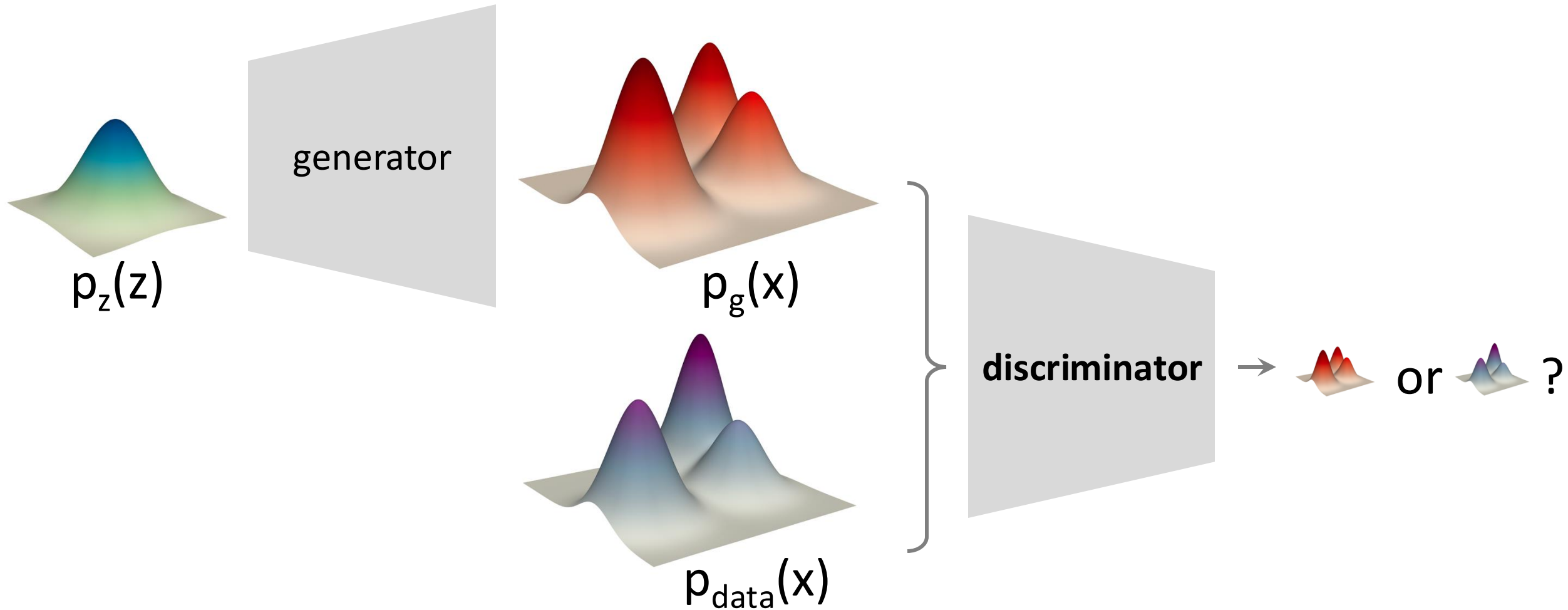
# Generative Adversarial Networks

Estimating **distribution difference** by neural network



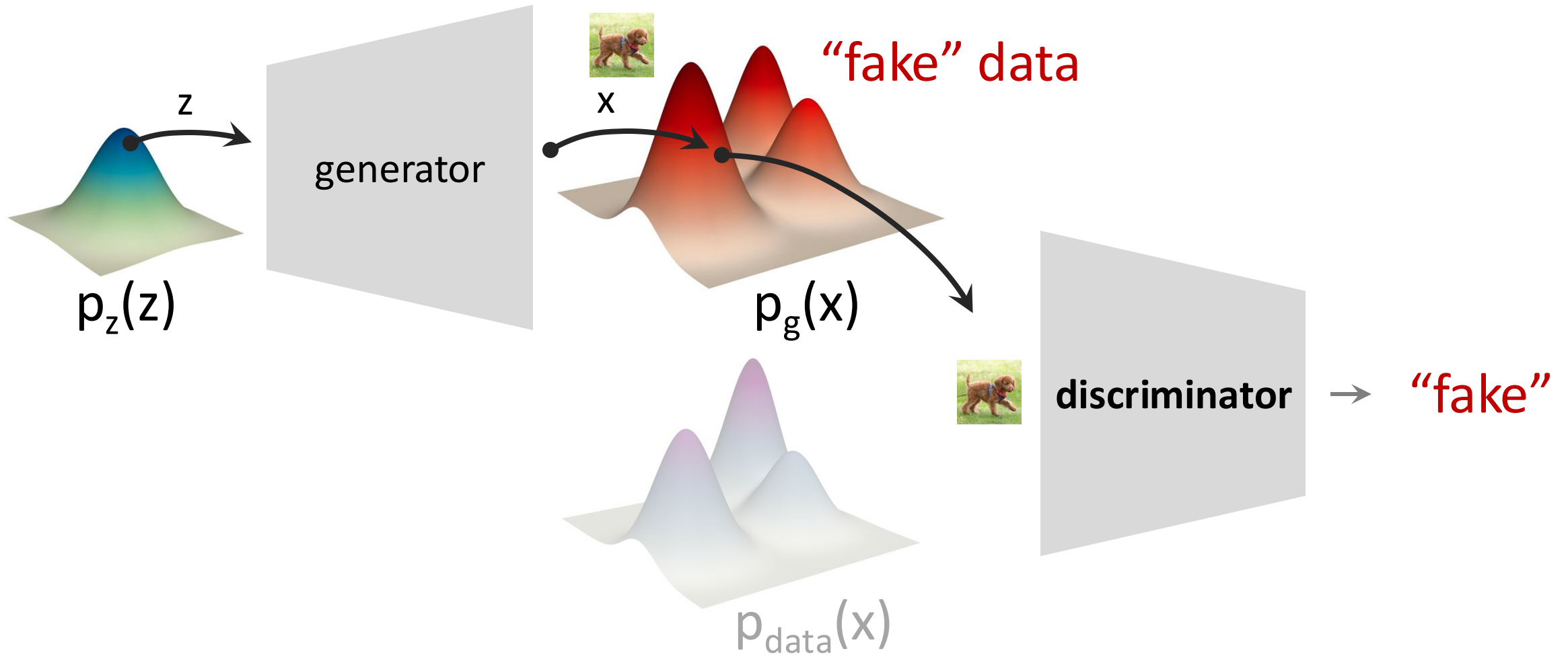
# Generative Adversarial Networks

Estimating **distribution difference** by neural network



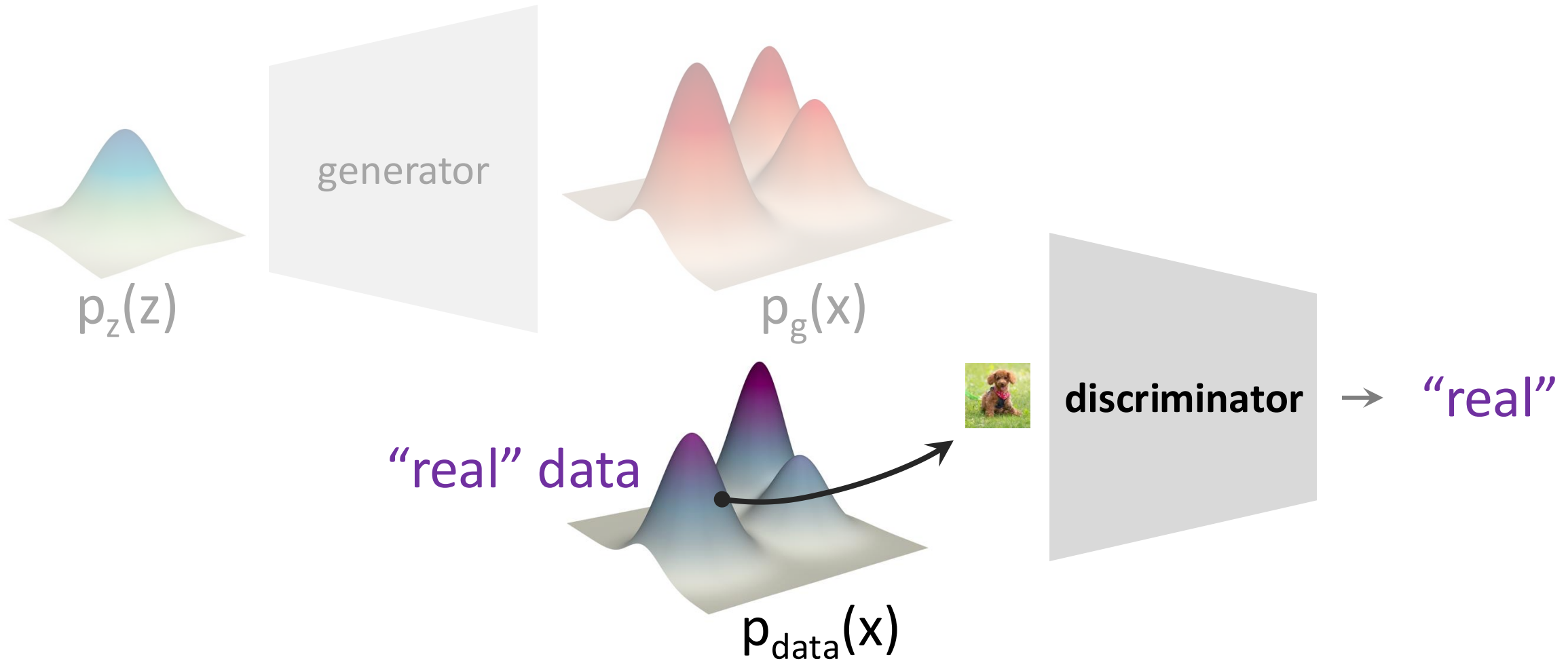
# Generative Adversarial Networks

Estimating **distribution difference** by neural network



# Generative Adversarial Networks

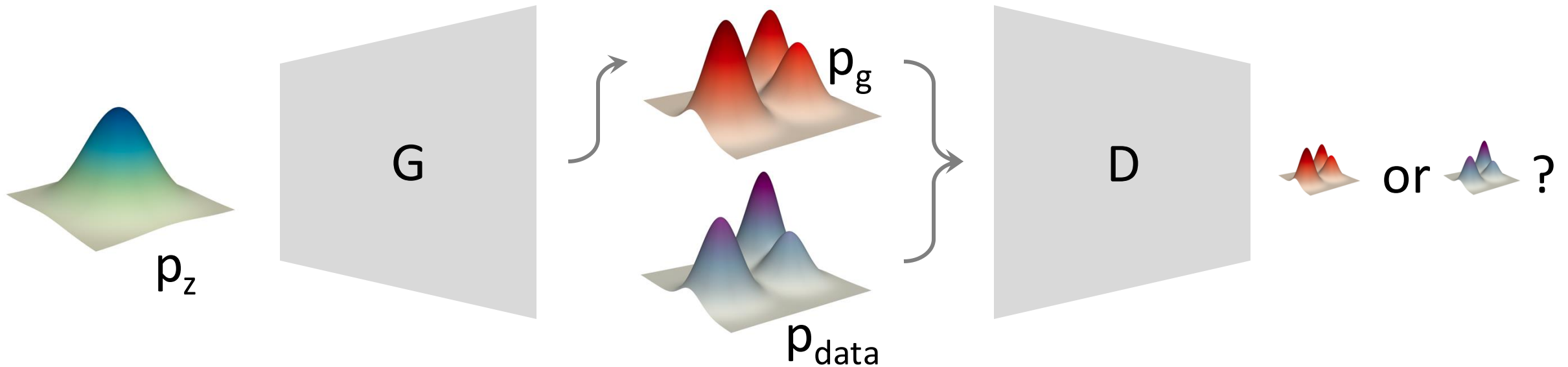
Estimating **distribution difference** by neural network



# Adversarial Objective

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

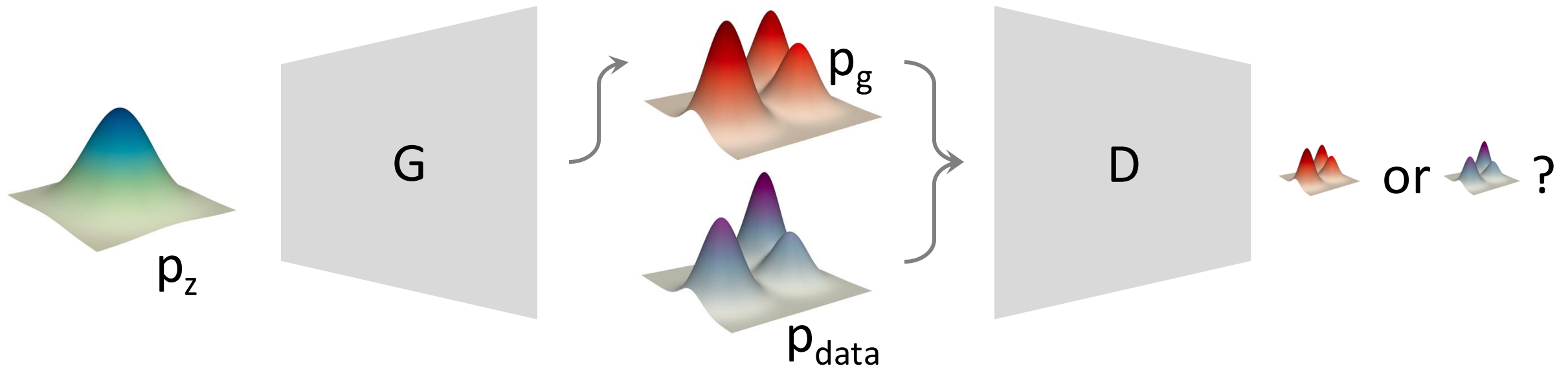
min-max process



# Adversarial Objective: D-step

$$\min_G \max_D \mathcal{L}(\underline{D}, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \underline{D}(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - \underline{D}(G(z)))]$$

D-step: fix G, optimize D

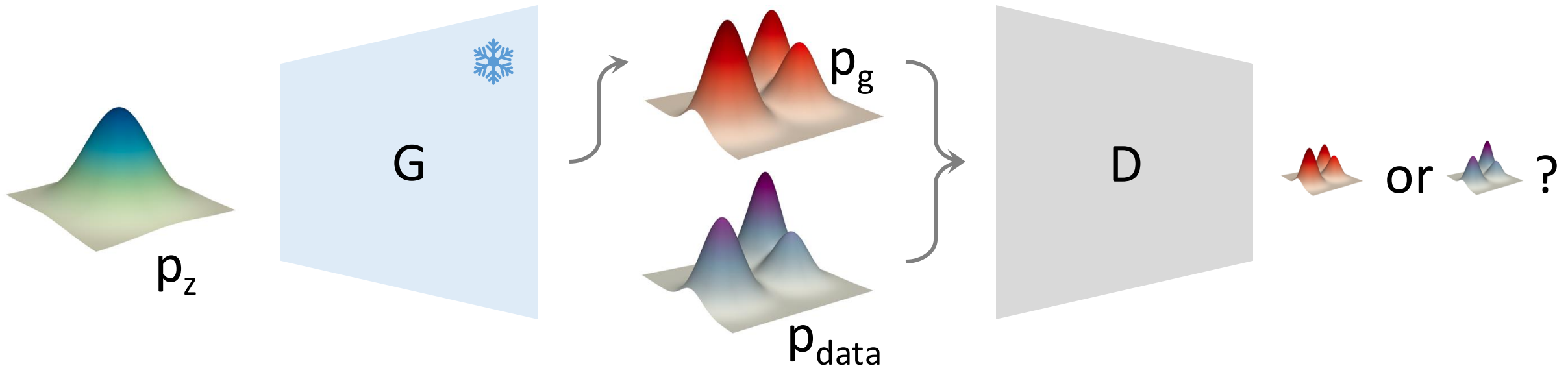


# Adversarial Objective: D-step

$$\max_D \mathcal{L}(D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \underbrace{D(x)}_{\text{push to 1}}] + \mathbb{E}_{z \sim p_z} [\log(1 - \underbrace{D(G(z))}_{\text{push to 0}})]$$

D-step: fix G, optimize D

- D : classify real or fake
- binary logistic regression (sigmoid + cross-entropy)



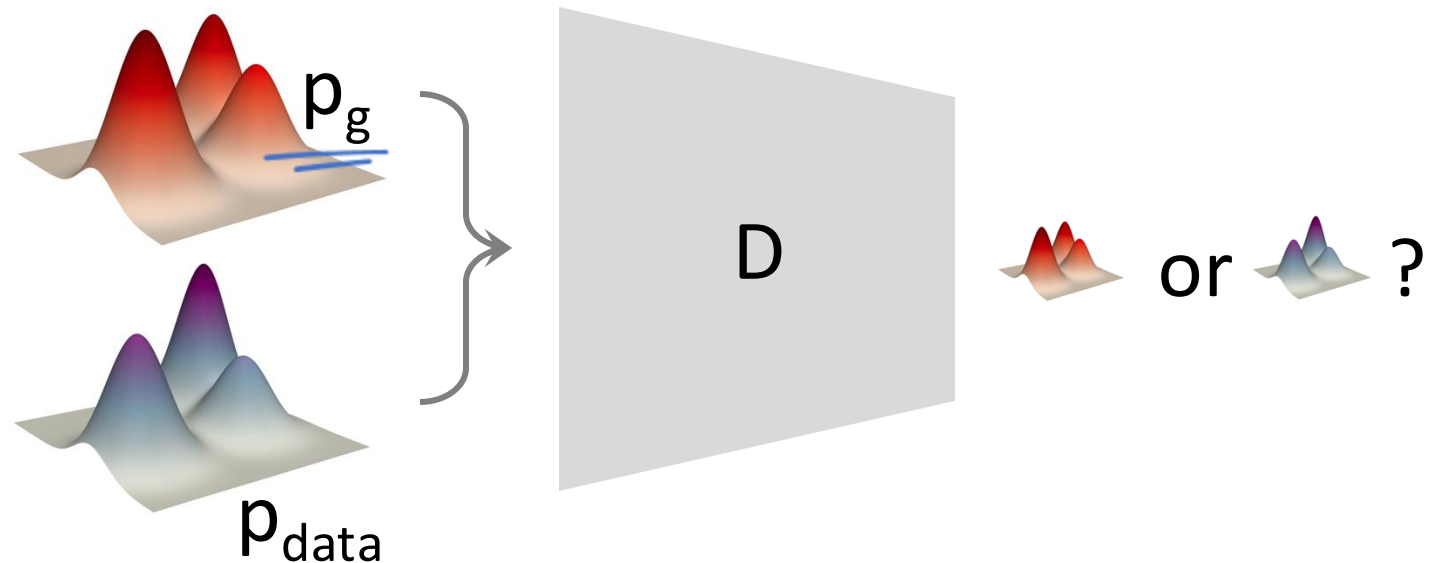
# Adversarial Objective: D-step

$$\max_D \mathcal{L}(D) = \mathbb{E}_{x \sim p_{\text{data}}} [\log \underbrace{D(x)}_{\text{push to 1}}] + \mathbb{E}_{x \sim \underline{p_g}} [\log(1 - \underbrace{D(x)}_{\text{push to 0}})]$$

=> sample x from  $p_g$

D-step: fix G, optimize D

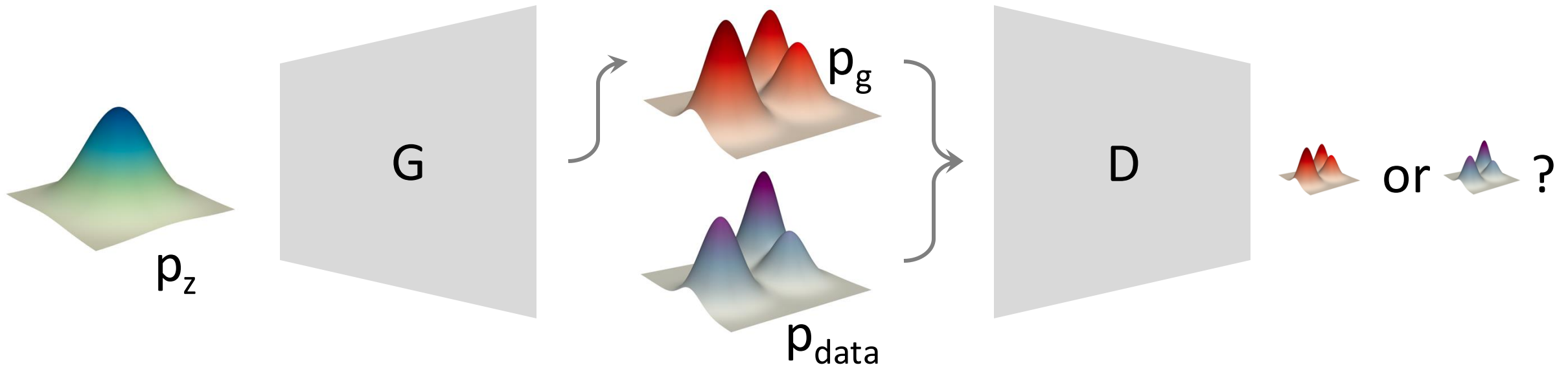
- D : classify real or fake
- binary logistic regression (sigmoid + cross-entropy)



# Adversarial Objective: G-step

$$\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

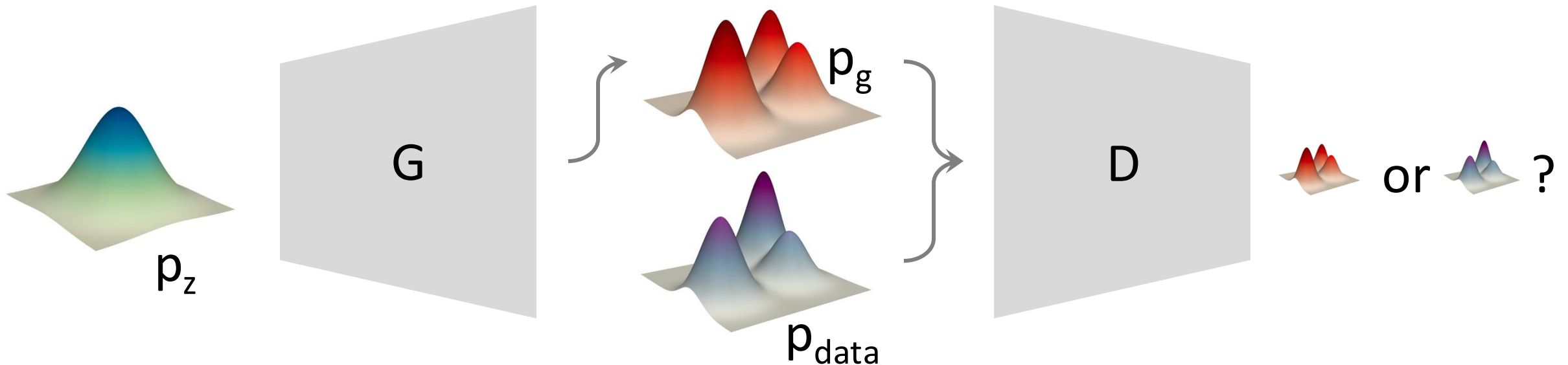
G-step: fix D, optimize G



# Adversarial Objective: G-step

$$\min_G \max_{\cancel{D}} \mathcal{L}(\cancel{D}, G) = \cancel{\mathbb{E}_{x \sim p_{\text{data}}} [\log \cancel{D}(x)]} + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

G-step: fix D, optimize G

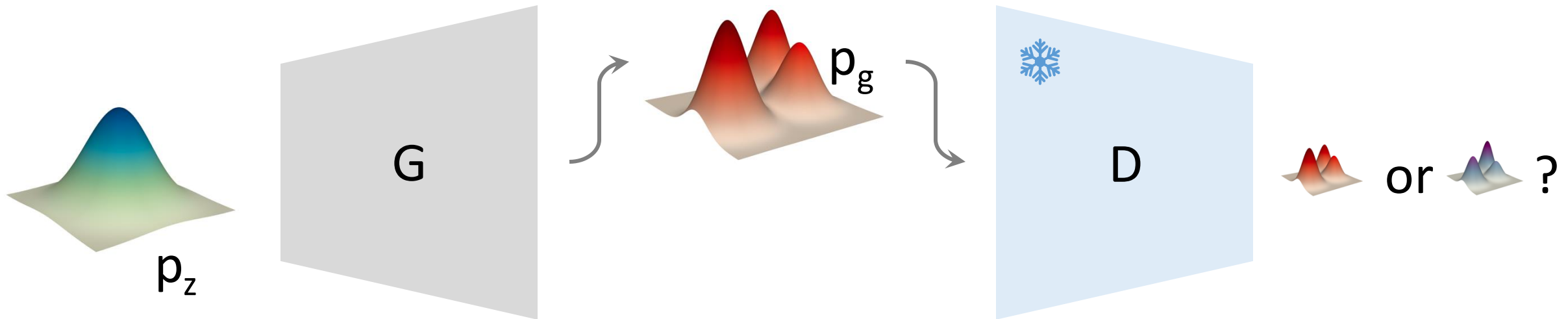


# Adversarial Objective: G-step

$$\min_G \mathcal{L}(G) = \mathbb{E}_{z \sim p_z} [\log(1 - \underbrace{D(G(z))}_{\text{push to 1}})]$$

G-step: fix D, optimize G

- generate fake data such that D classifies it as “real”
- G to “confuse” D

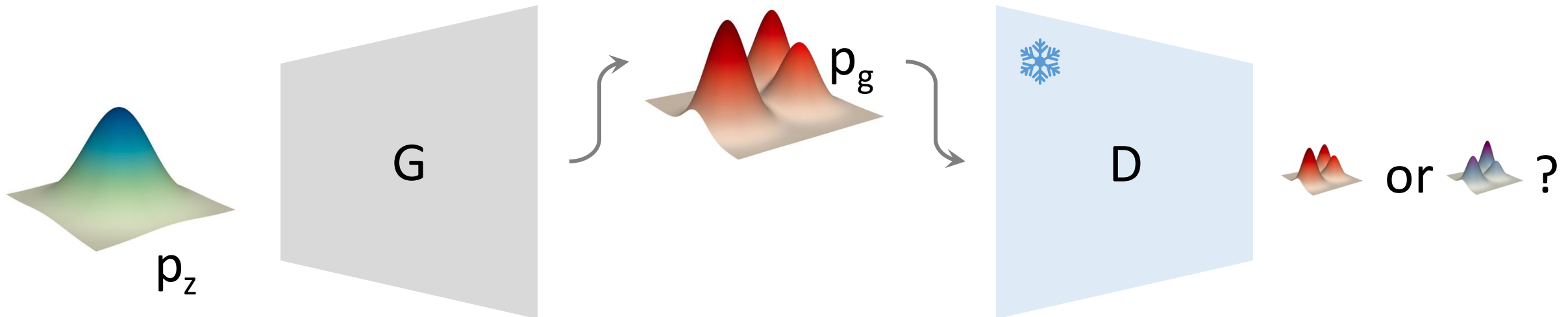


# Adversarial Objective: G-step

a “flip” trick:  $\max_G \mathcal{L}(G) = \mathbb{E}_{z \sim p_z} [\log(\underbrace{1 - D(G(z))}_{\text{push to 1}})]$

G-step: fix D, optimize G

- generate fake data such that D classifies it as “real”
- G to “confuse” D



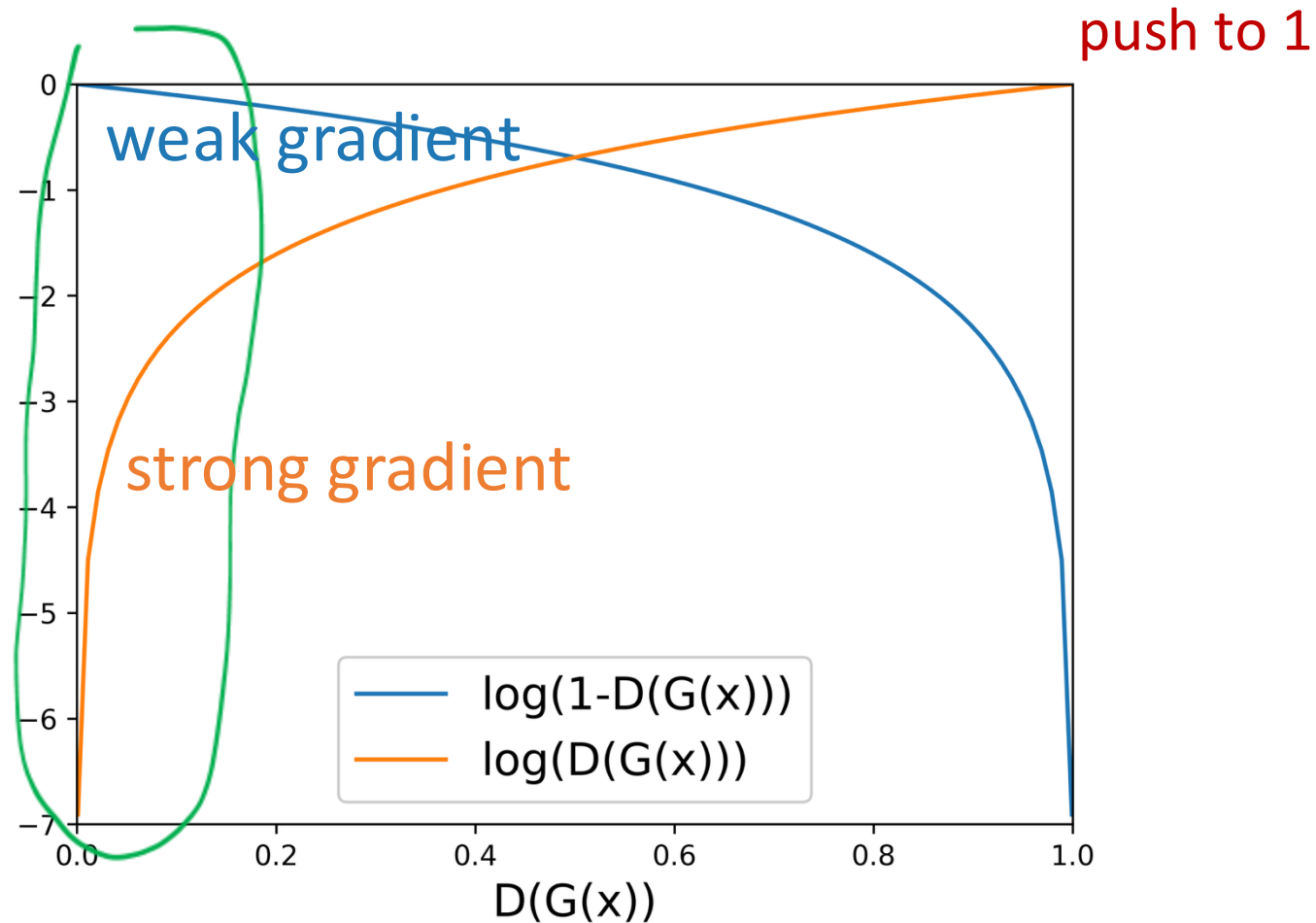
# Adversarial Objective: G-step

a “flip” trick:

$$\max_G \mathcal{L}(G) = \mathbb{E}_{z \sim p_z} [\log(1 - \underbrace{D(G(z))}_{\text{push to 1}})]$$

Early in training:

- G is poor
- D(G) is near 0



# GAN algorithm annotated

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

## minibatch SGD

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

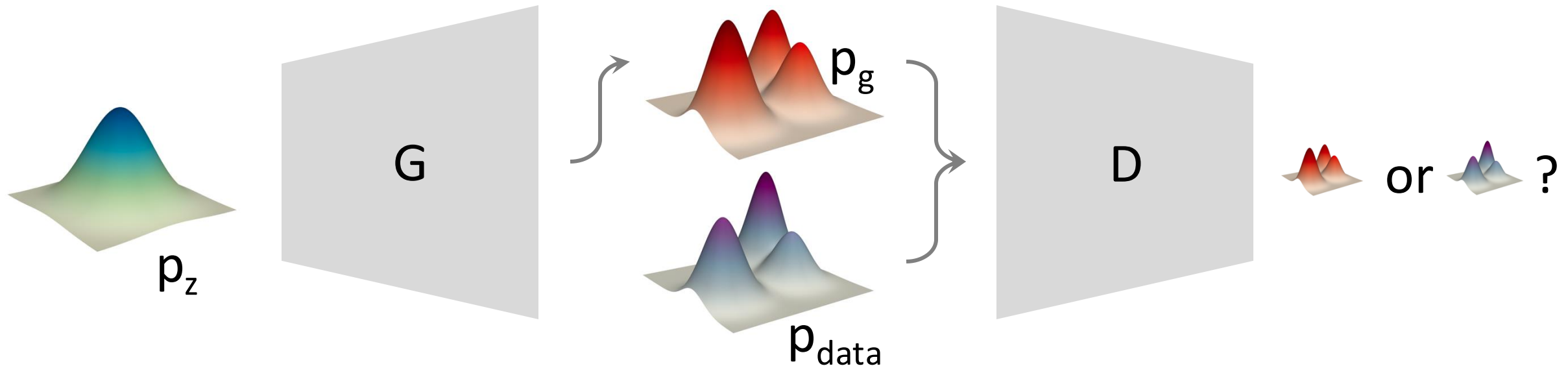
**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



# GAN algorithm annotated

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

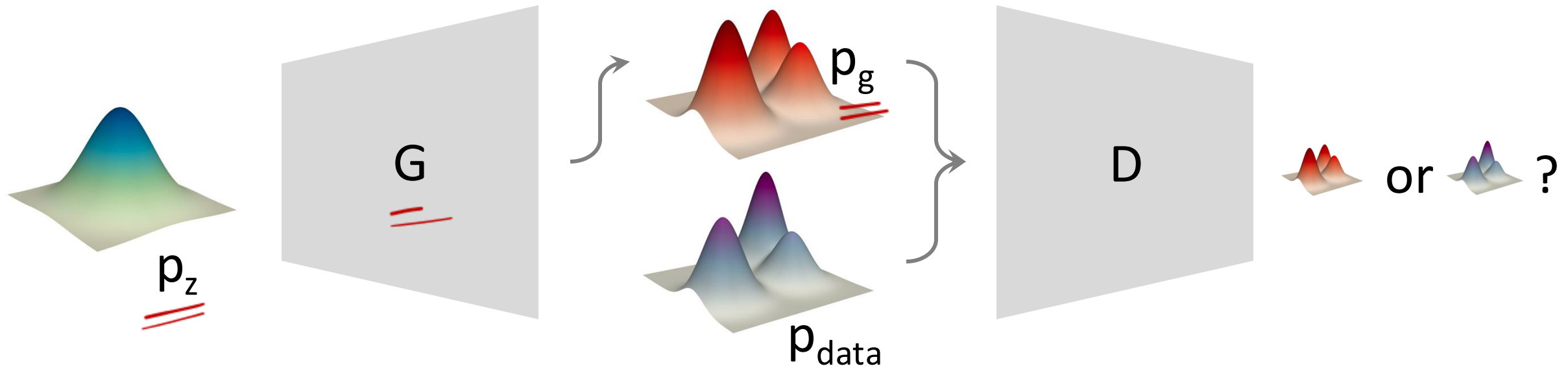
**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



# GAN algorithm annotated

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(\underline{x^{(i)}}) + \log (1 - D(G(z^{(i)}))) \right].$$

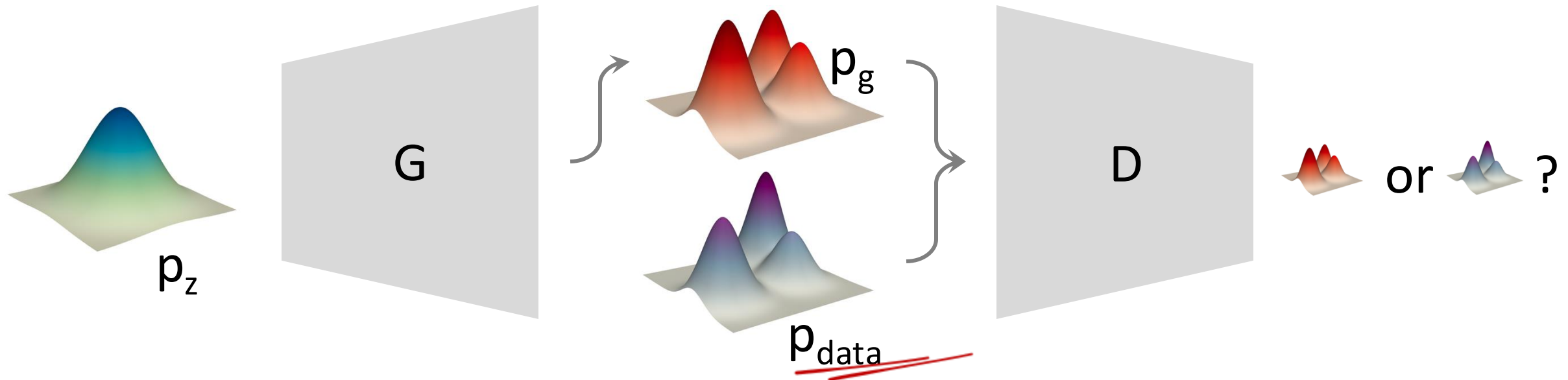
**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .

- Update the discriminator by ascending its stochastic gradient:

**D-step** 
$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

**end for**

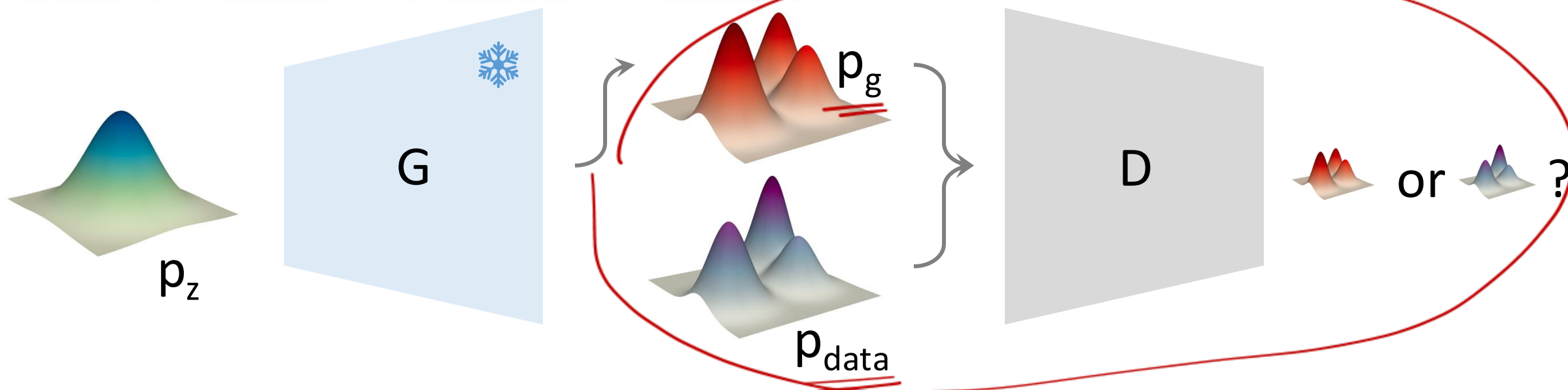
- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

gradient **ascend**  
(**maximize**)

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



# GAN algorithm annotated

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

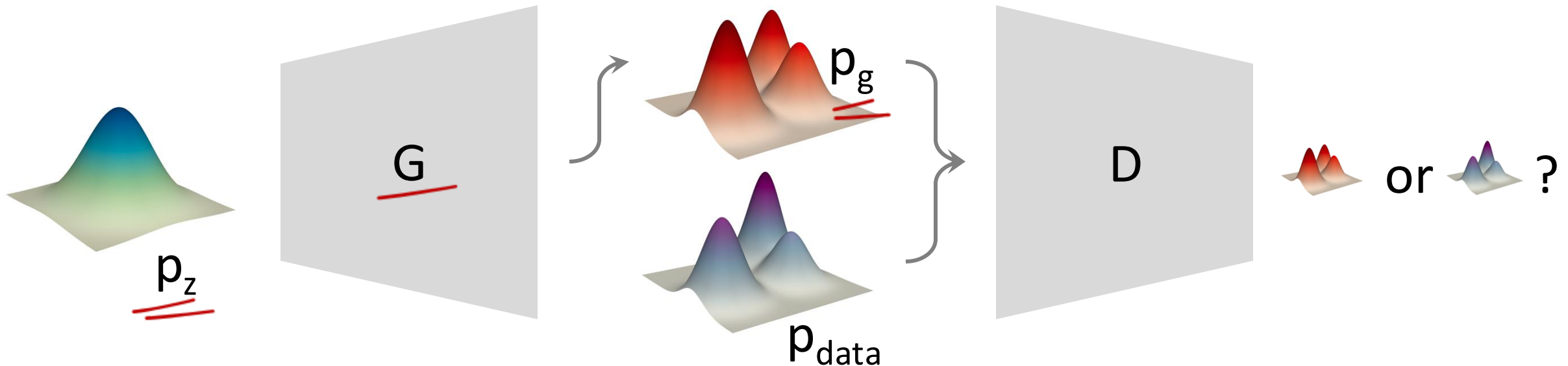
**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



# GAN algorithm annotated

**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

**G-step**

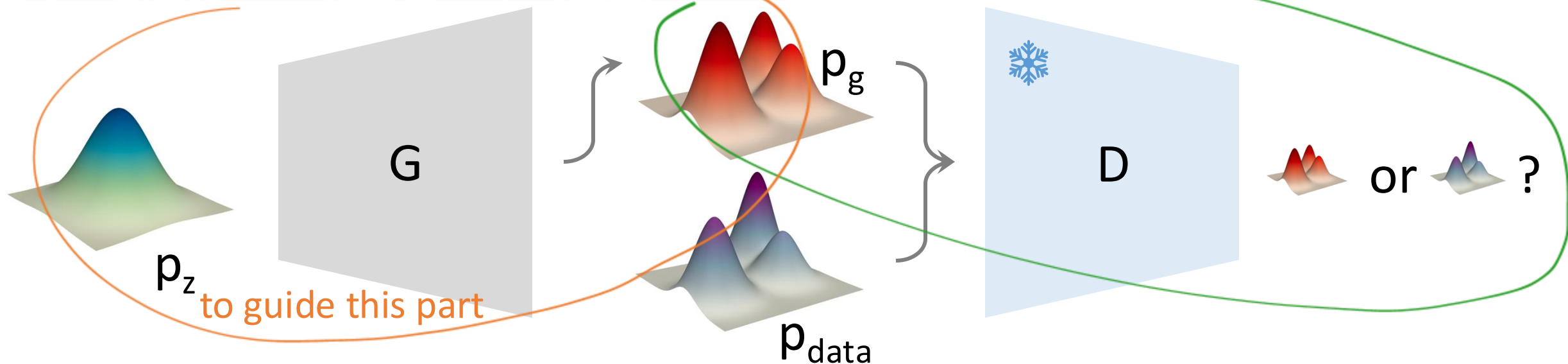
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**gradient descend  
(minimize)**

**end for**

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

a parameterized  
loss function



**Algorithm 1** Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator,  $k$ , is a hyperparameter. We used  $k = 1$ , the least expensive option, in our experiments.

**for** number of training iterations **do**

**for**  $k$  steps **do**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Sample minibatch of  $m$  examples  $\{x^{(1)}, \dots, x^{(m)}\}$  from data generating distribution  $p_{\text{data}}(x)$ .
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[ \log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

**end for**

- Sample minibatch of  $m$  noise samples  $\{z^{(1)}, \dots, z^{(m)}\}$  from noise prior  $p_g(z)$ .
- Update the generator by descending its stochastic gradient:

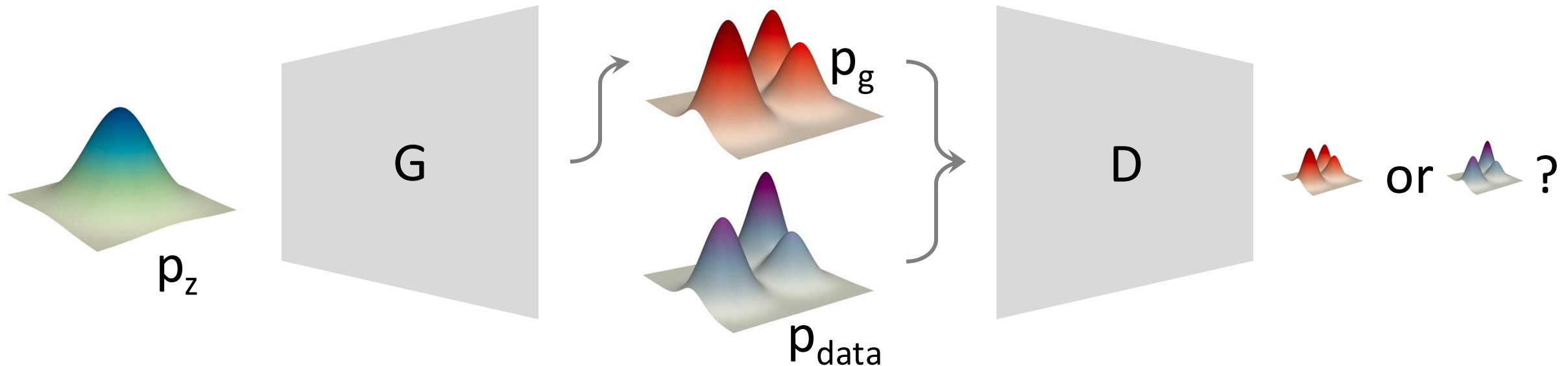
$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

**end for**

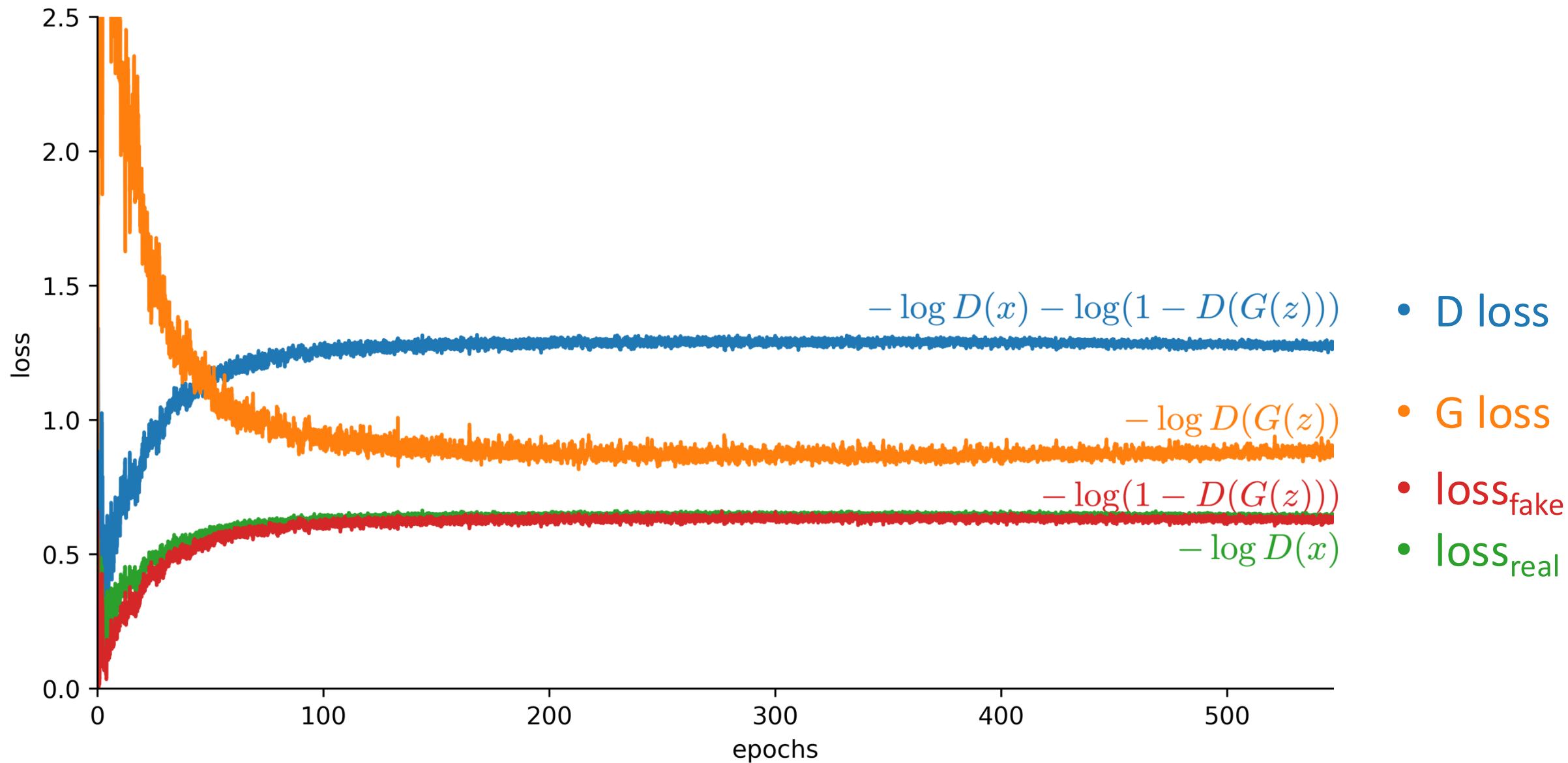
The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

# GAN algorithm annotated

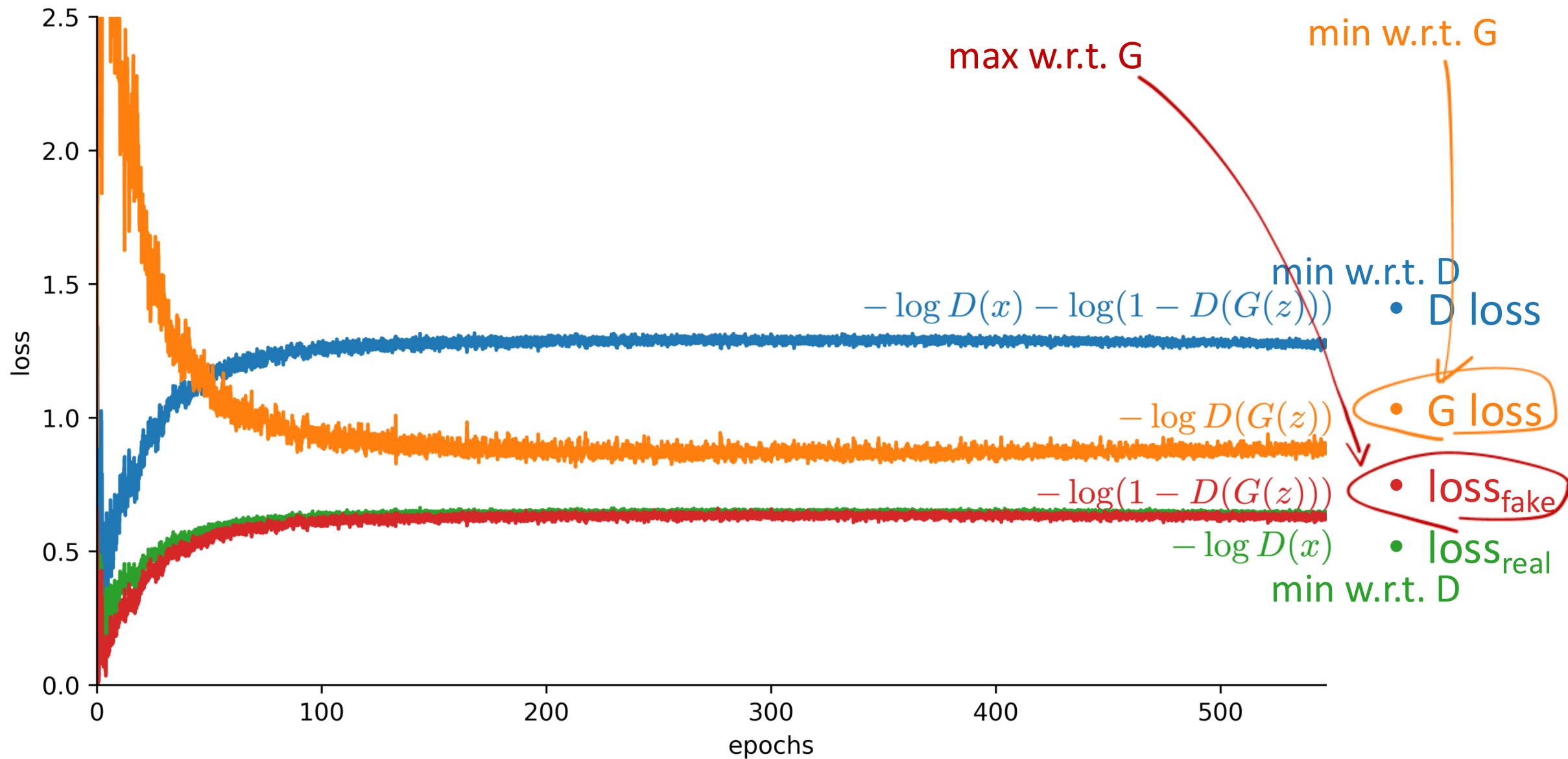
iterating  
min-max



# Running example: MNIST



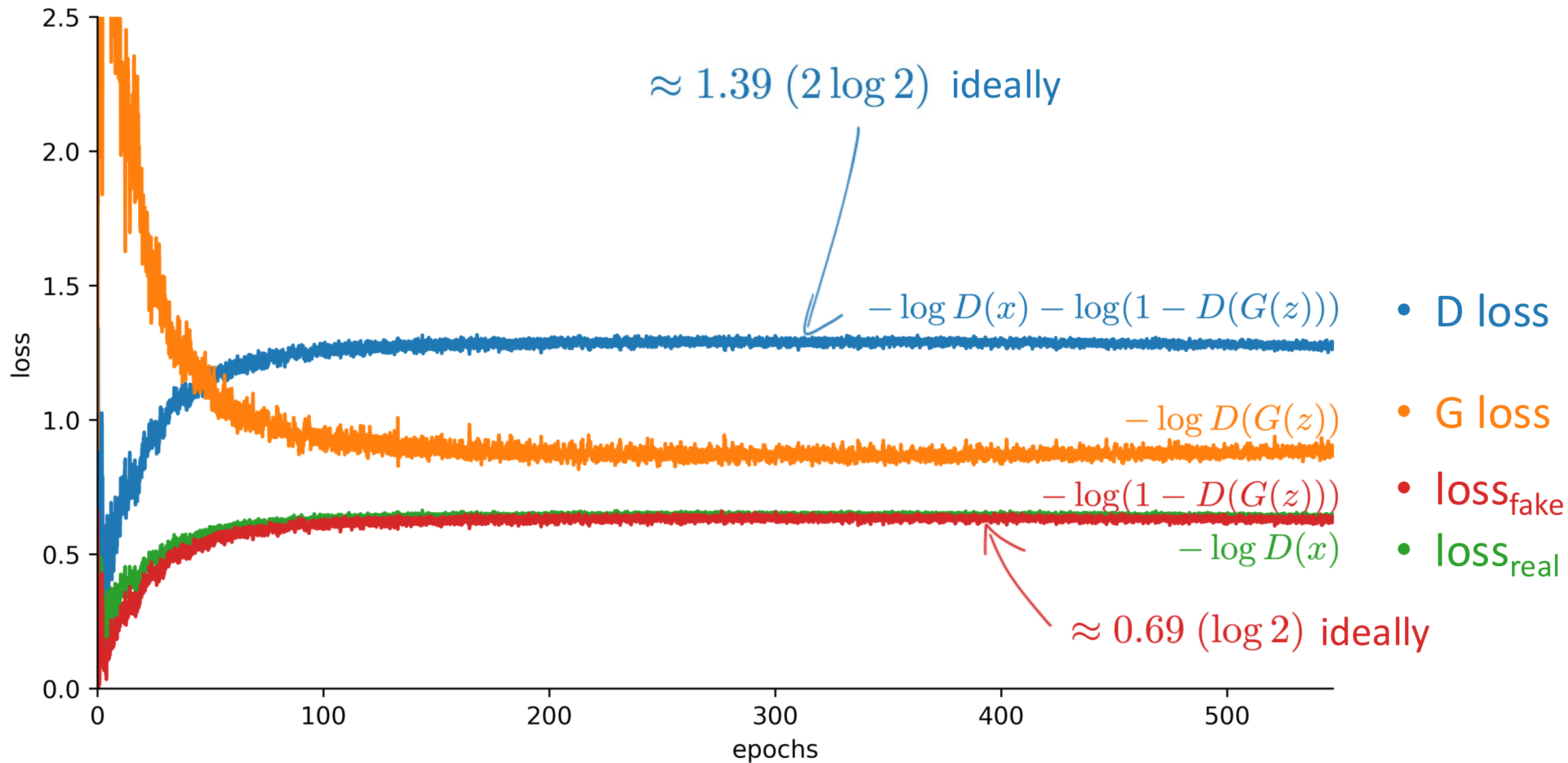
# Running example: MNIST



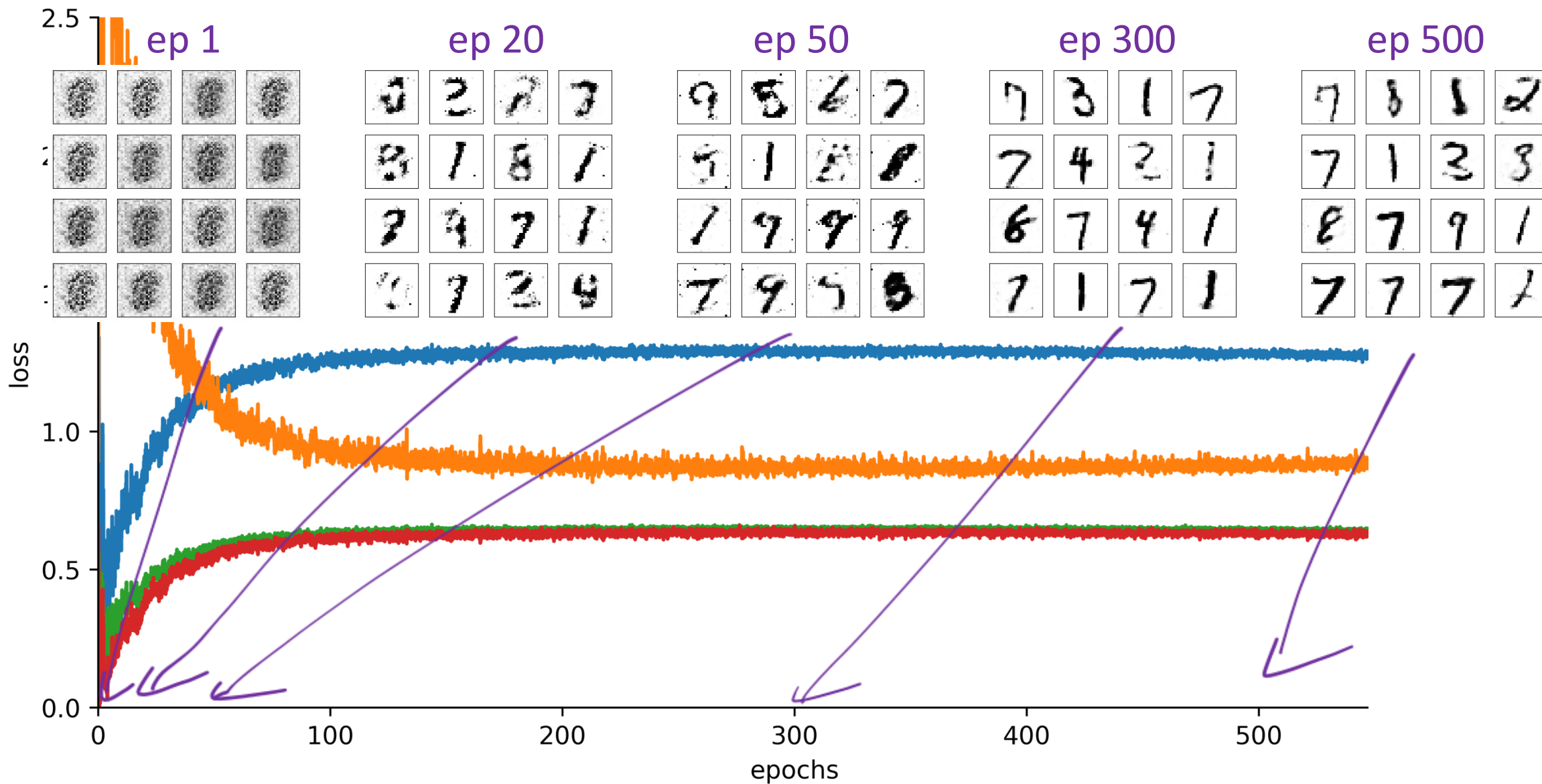
\*All objectives are negative of their original form

Code adapted from: <https://github.com/prcastro/pytorch-gan/tree/master>

# Running example: MNIST

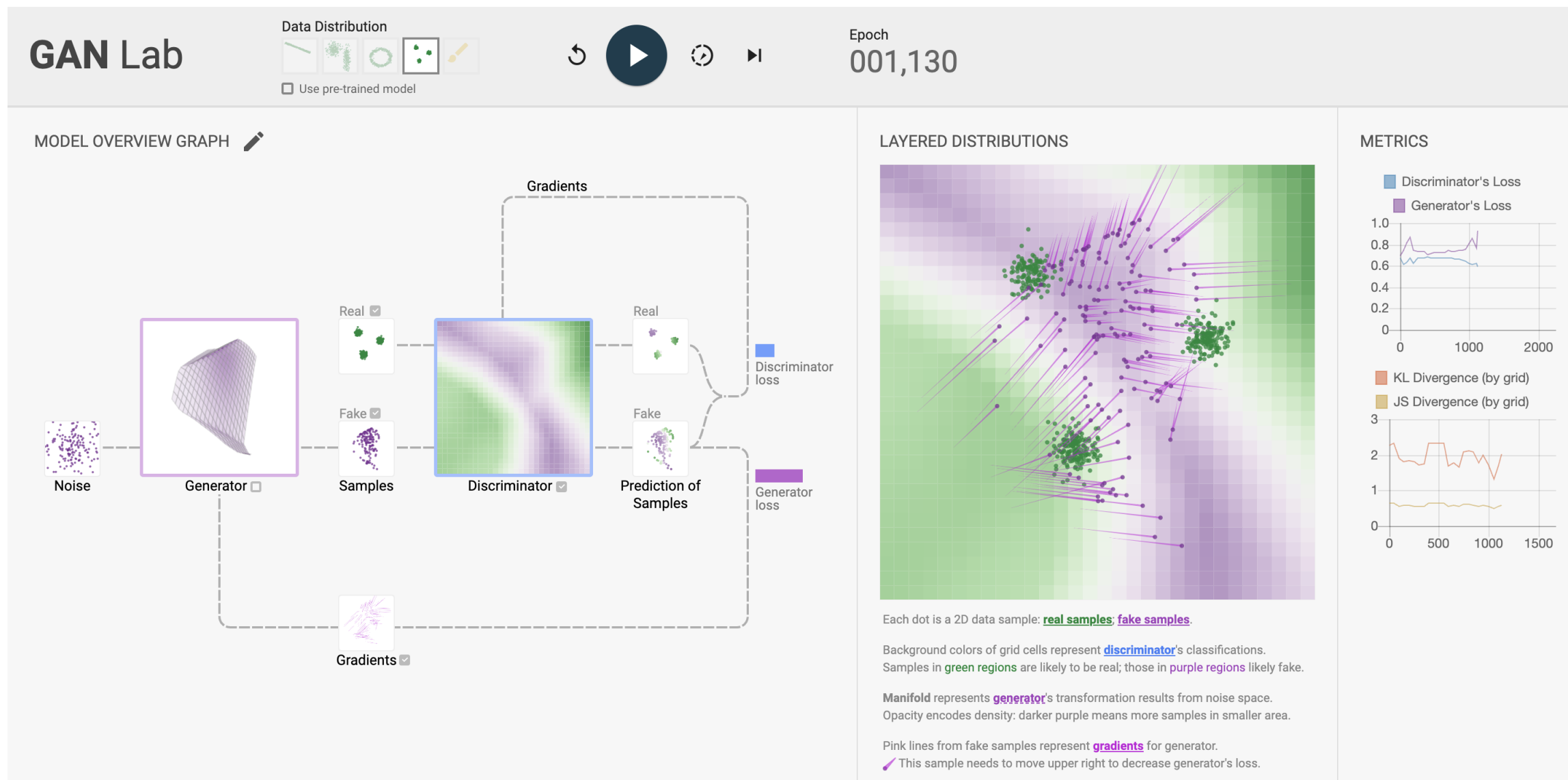


# Running example: MNIST



# Running example: GAN Lab

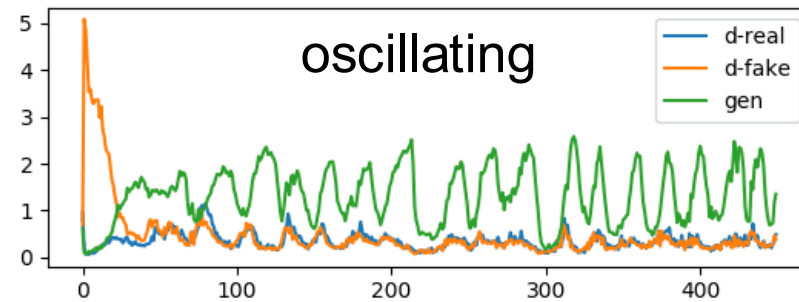
<https://poloclub.github.io/ganlab/>



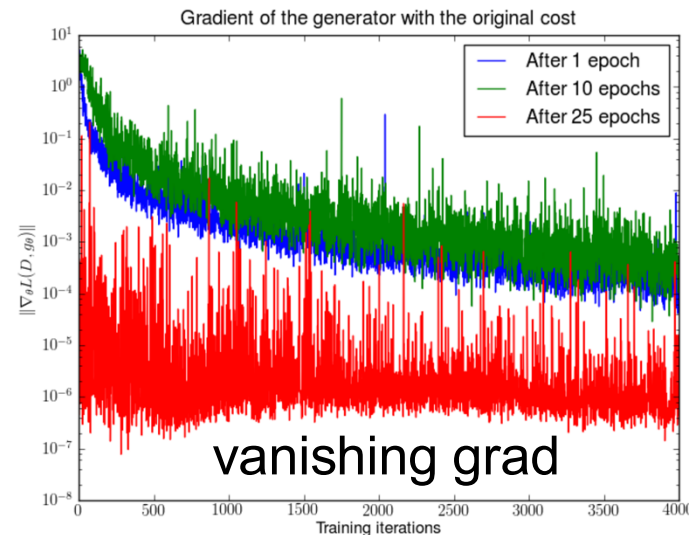
# Issues of GAN

Difficult to train/converge

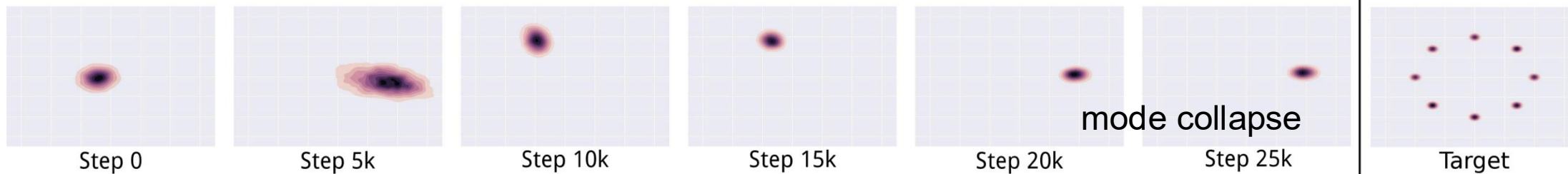
- Hard to achieve equilibrium
- Vanishing gradients
- Mode collapse



J. Brownlee, "How to Identify and Diagnose GAN Failure Modes"



Arjovsky & Bottou, "Towards Principled Methods for Training GANs"



L. Metz, "Unrolled Generative Adversarial Networks"

# Adversary as a Loss Function

- GAN essentially defines an **adversarial loss** function
- Input to networks is **not** necessarily random/noise
- **Beyond L2/L1**: adversarial loss encourages output to look “realistic”
- **Combined with L2/L1**: reconstruction loss largely stabilizes training

# Example: Super-Resolution GAN

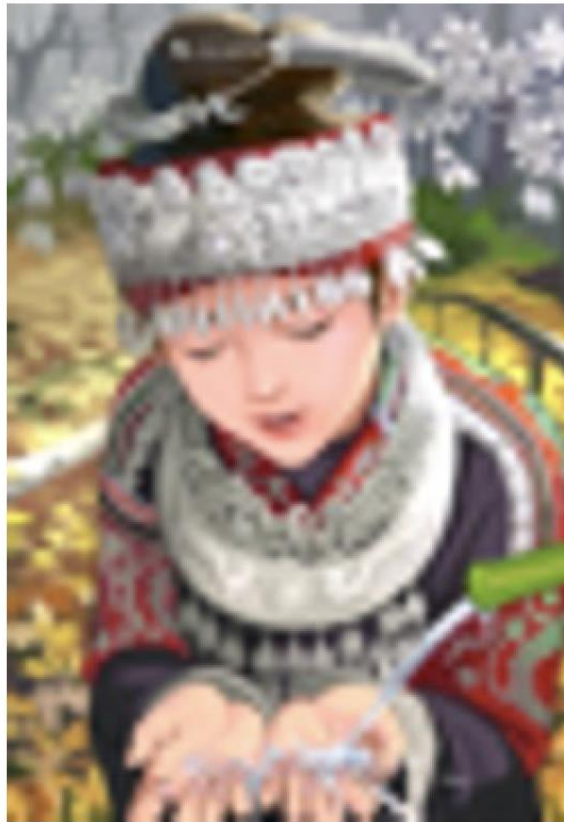
- **L2 loss: better PSNR**

- **Adv loss: better visual quality**

original



bicubic  
(21.59dB/0.6423)



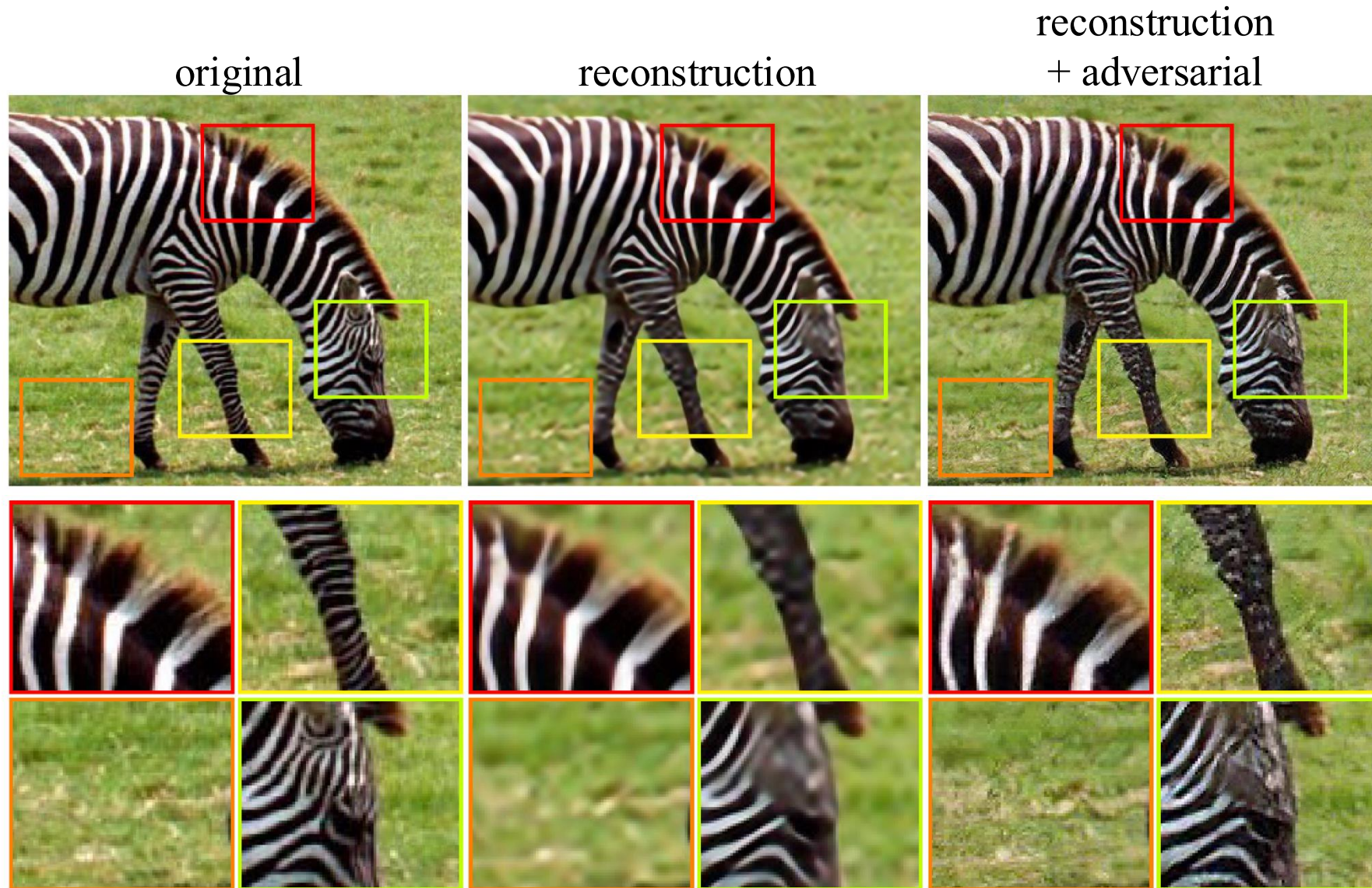
SRResNet  
(23.44dB/0.7777)



SRGAN  
(20.34dB/0.6562)



# Example: Super-Resolution GAN



# Example: VQ-VAE to VQ-GAN

- Today's most popular image **tokenizer** for Diffusion or AR

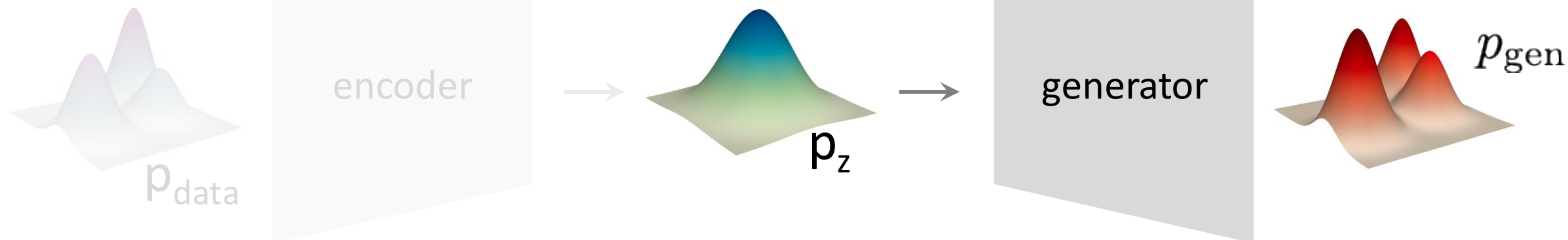
VQ-VAE (L2)

VQ-GAN (L2 + adv)

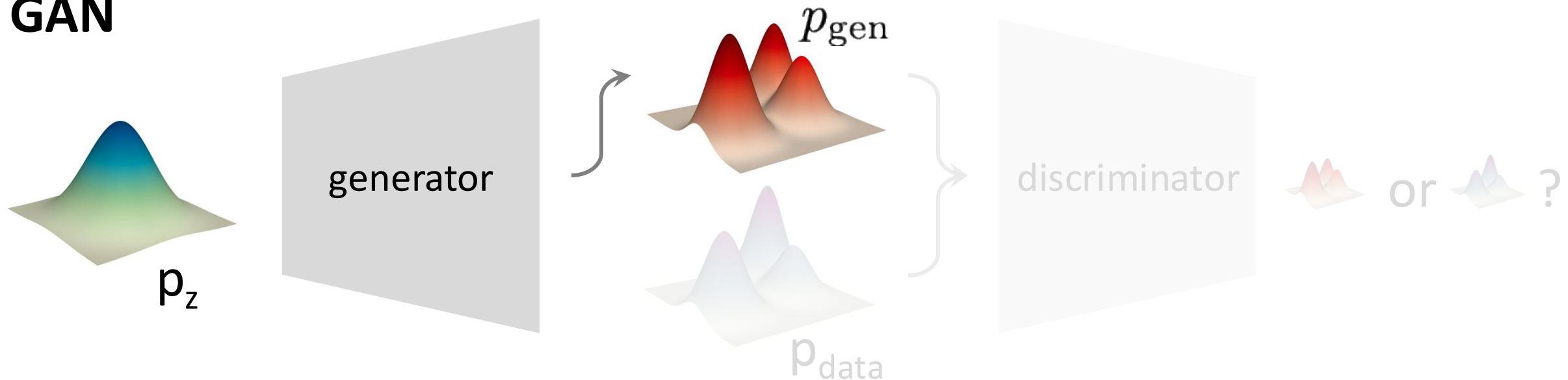


## VAE

- VAE and GAN: **one-step** generation
- Diffusion, AR: **multi-step** generation (next lecture)



## GAN



# This Lecture

- Latent Variable Models
- Variational Autoencoder (VAE)
- Generative Adversarial Network (GAN)